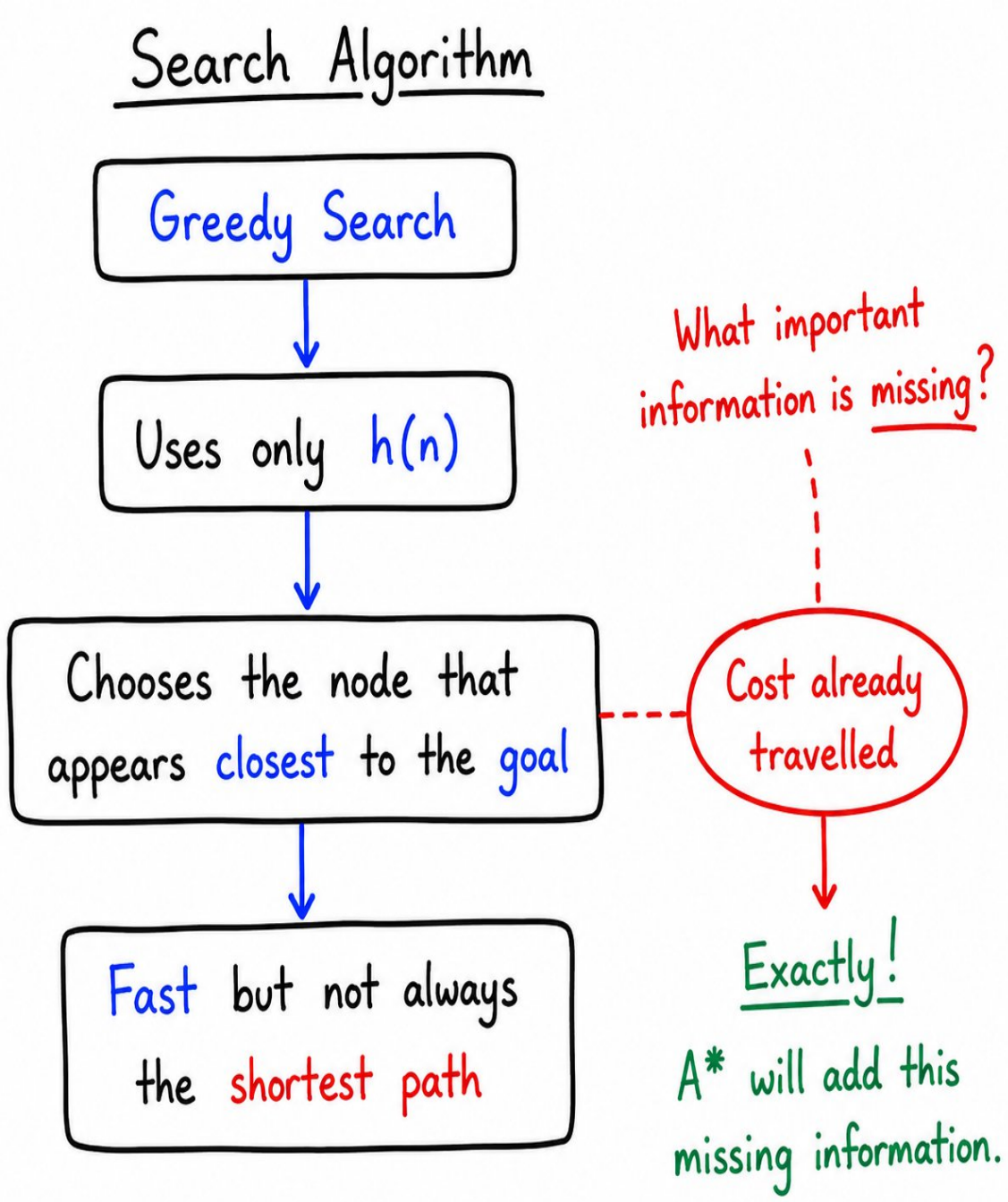
# Informed Search - II

A* Search Algorithm



# Previous Lecture

- Heuristics
  - An estimate of the remaining cost from the current node to the goal.
  - Used to guide the search toward promising paths.
- Greedy Search
  - Selects the node with the smallest heuristic value **h(n)**.
  - Fast because it always moves toward the estimated goal.
  - Does **not** guarantee the shortest path.



# A* Search

eval-fn: $f(n) = g(n) + h(n)$

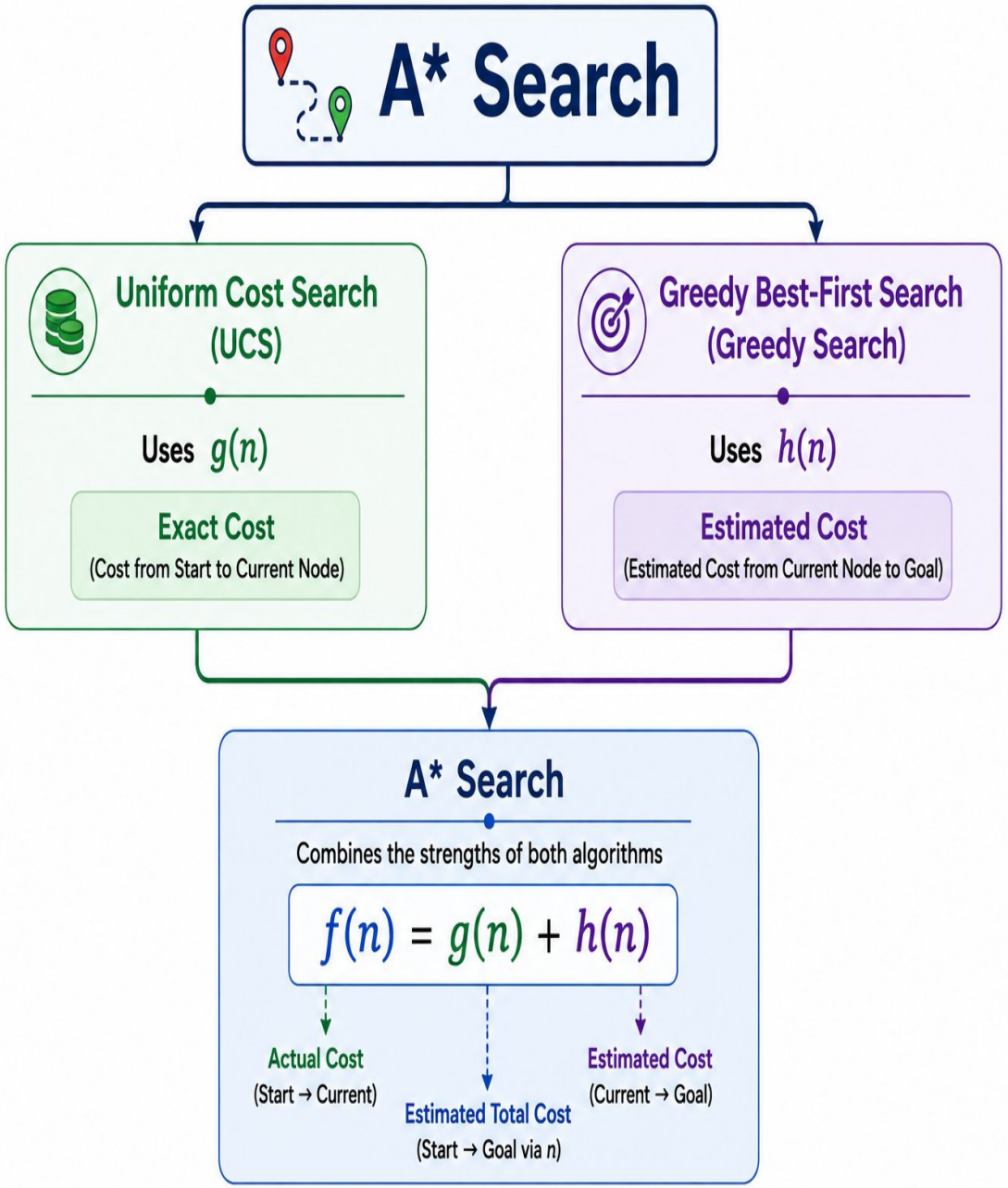
# Need for A*? (A Star)

- Greedy Best First Search
  - Fast because it uses heuristic information
  - May find a non-optimal path
  - Not always complete
- Uniform Cost Search
  - Finds the lowest-cost solution
  - Complete and Optimal
  - Explores many unnecessary nodes
  - Can be slow

- New Strategy: Combine Greedy Search and UCS to get an efficient algorithm which is complete and optimal.

- A* uses a heuristic function which combines g(n) and h(n):

$$f(n) = g(n) + h(n)$$

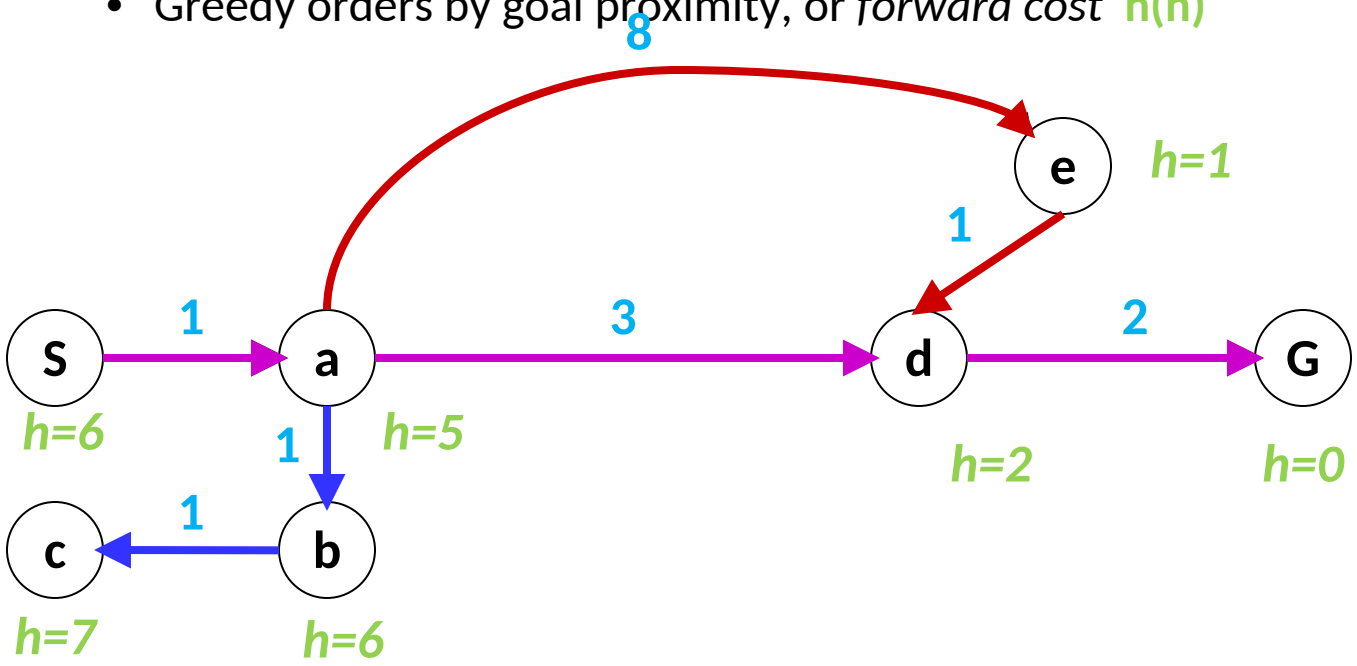


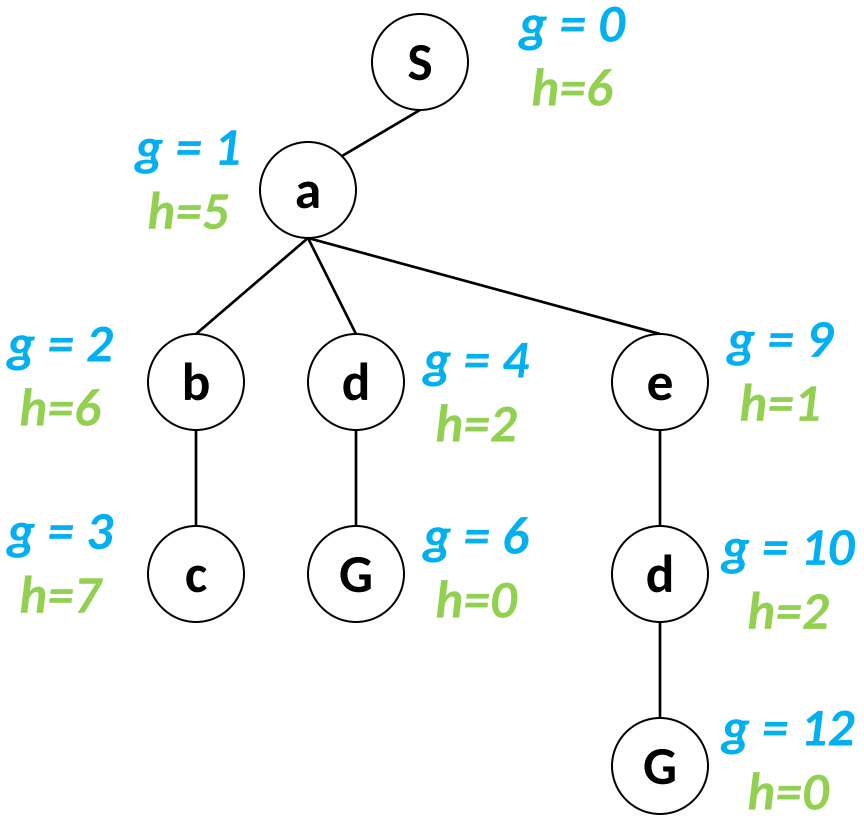
# Comparison Table

| Search Algorithm | Orders Nodes By | Uses | Main Idea |
|---|---|---|---|
| **Uniform Cost Search** | **g(n)** | Actual cost | Expand the cheapest path so far |
| **Greedy Best-First Search** | **h(n)** | Estimated cost | Expand the node that appears closest to the goal |
| **A*** | **f(n)=g(n)+h(n)** | Actual + Estimated cost | Expand the node with the lowest estimated total cost |

# How Does A* Make Better Decisions?

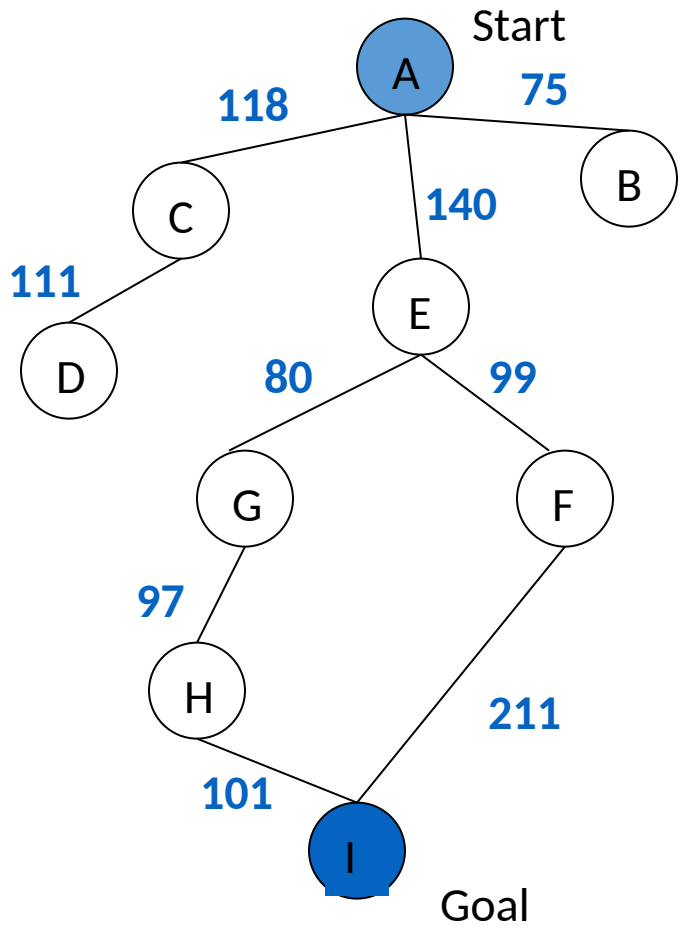
- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)




- A* Search orders by the sum: f(n) = g(n) + h(n)

# A* Search




| State | Heuristic: h(n) |
|---|---|
| A | 366 |
| B | 374 |
| C | 329 |
| D | 244 |
| E | 253 |
| F | 178 |
| G | 193 |
| H | 98 |
| I | 0 |

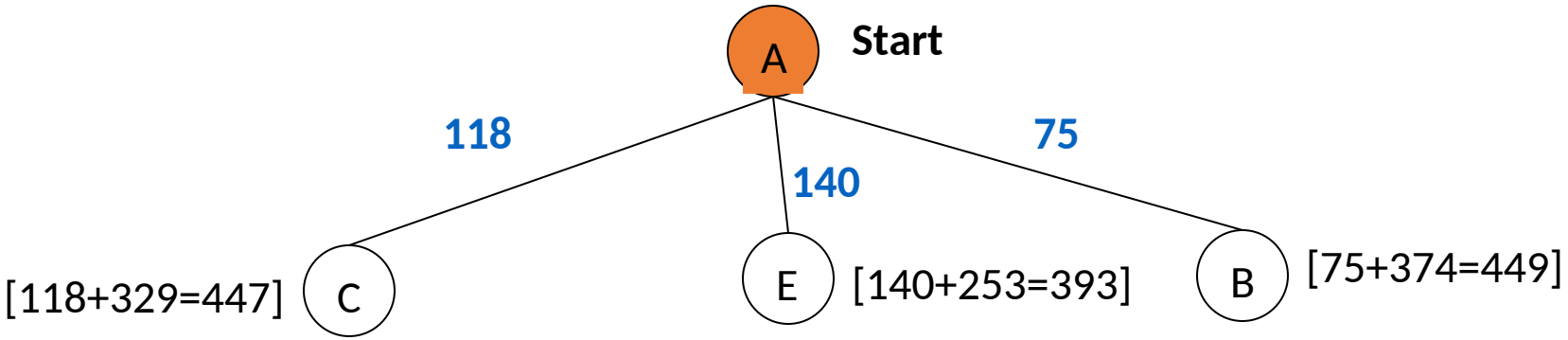
$$f(n) = g(n) + h(n)$$

**g(n):** is the exact cost to reach node *n* from the initial state.

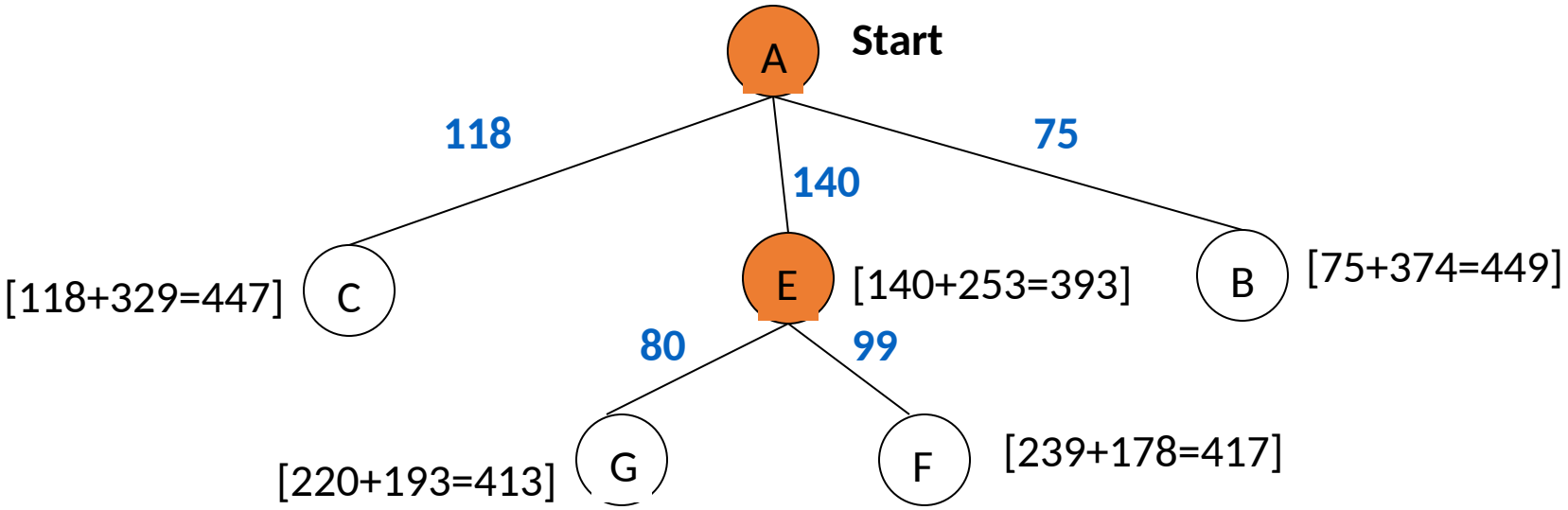
# A* Search: Tree Search



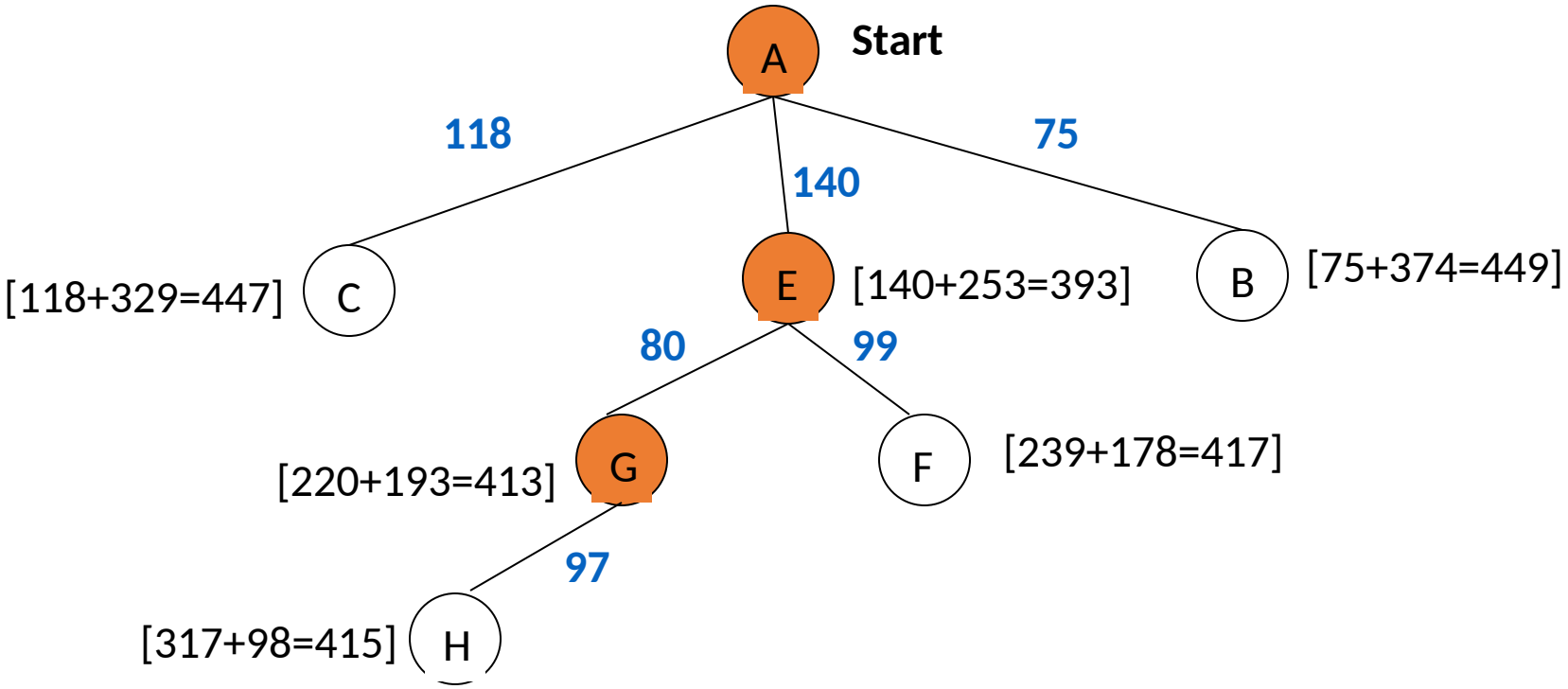
# A* Search: Tree Search



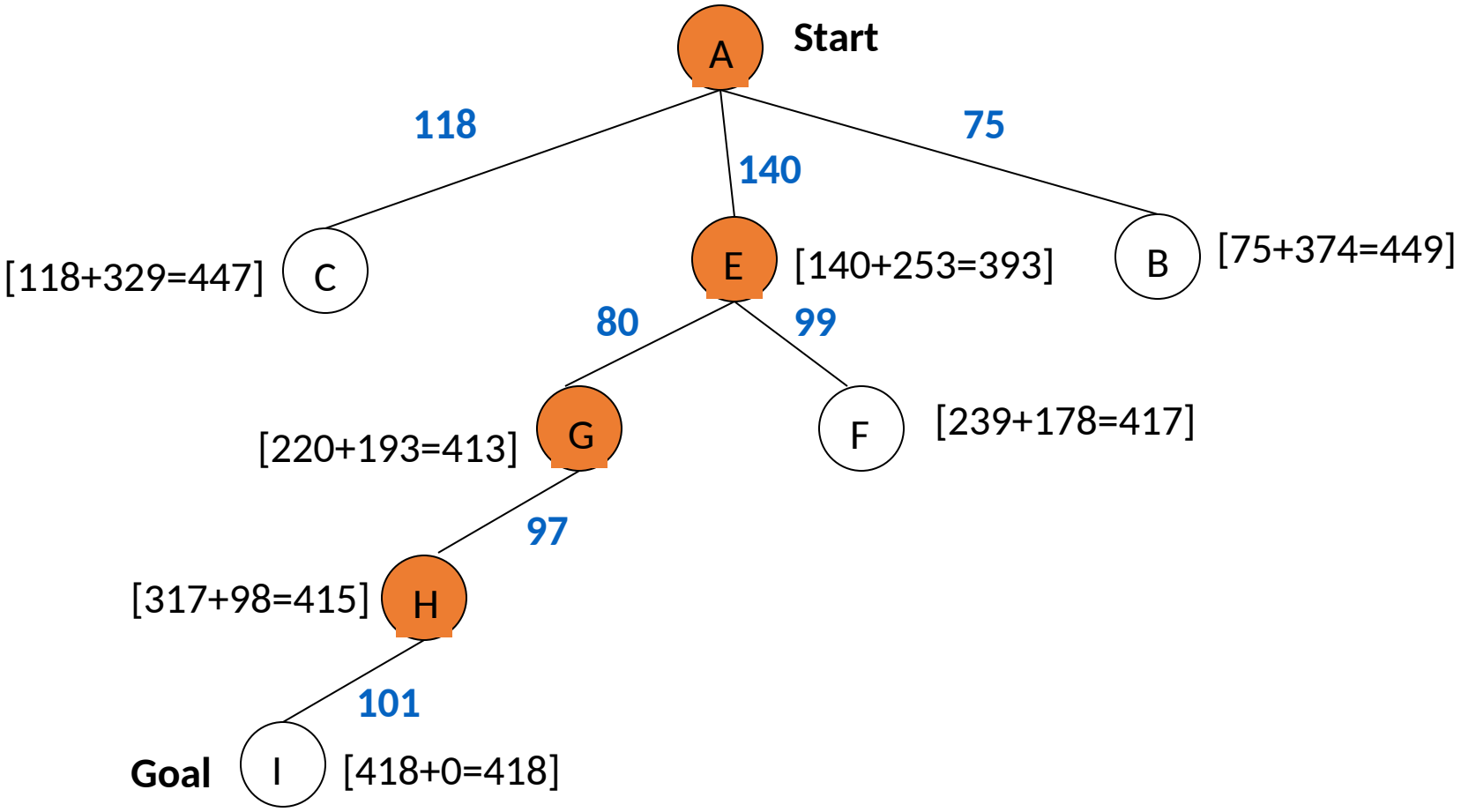
# A* Search: Tree Search



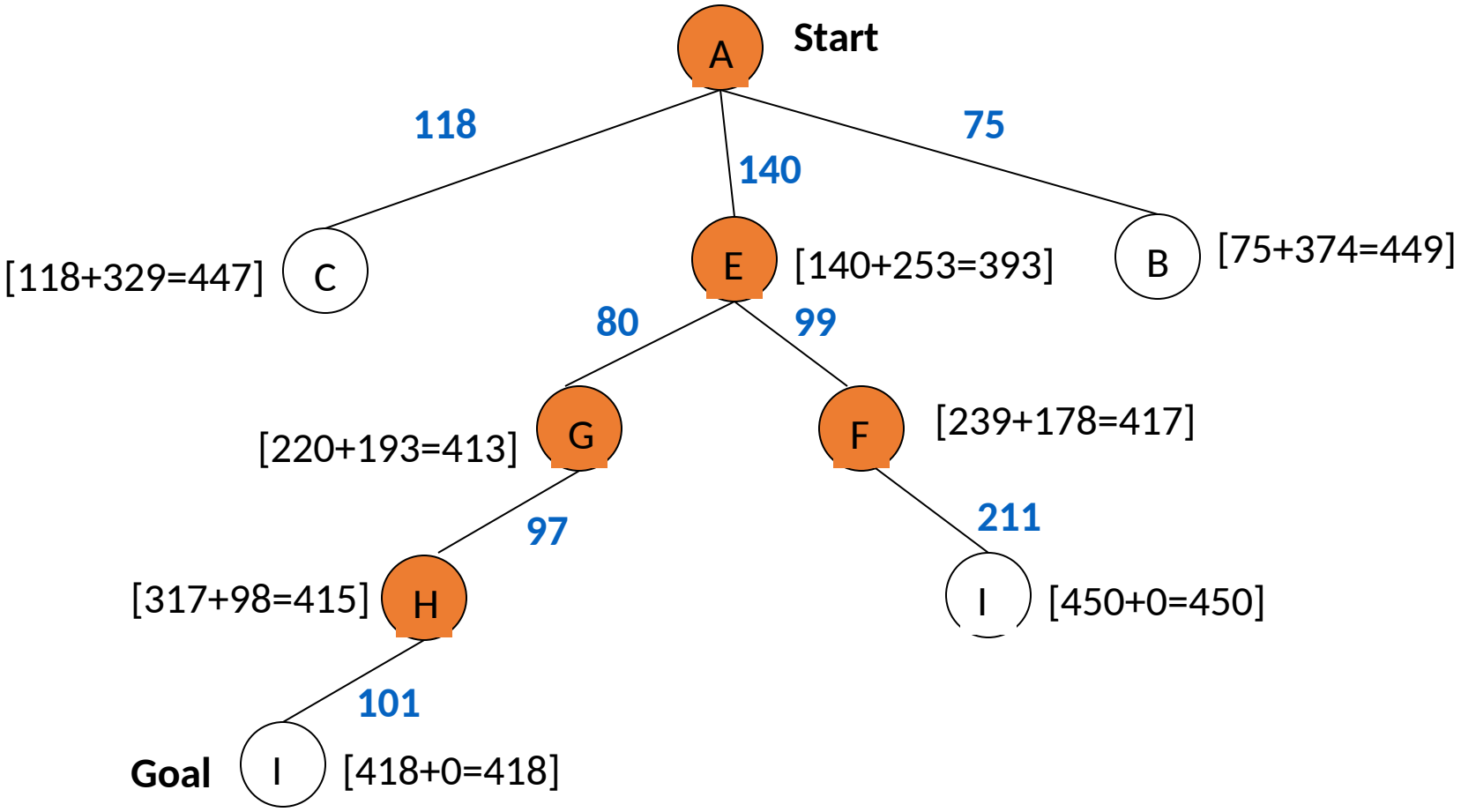
# A* Search: Tree Search



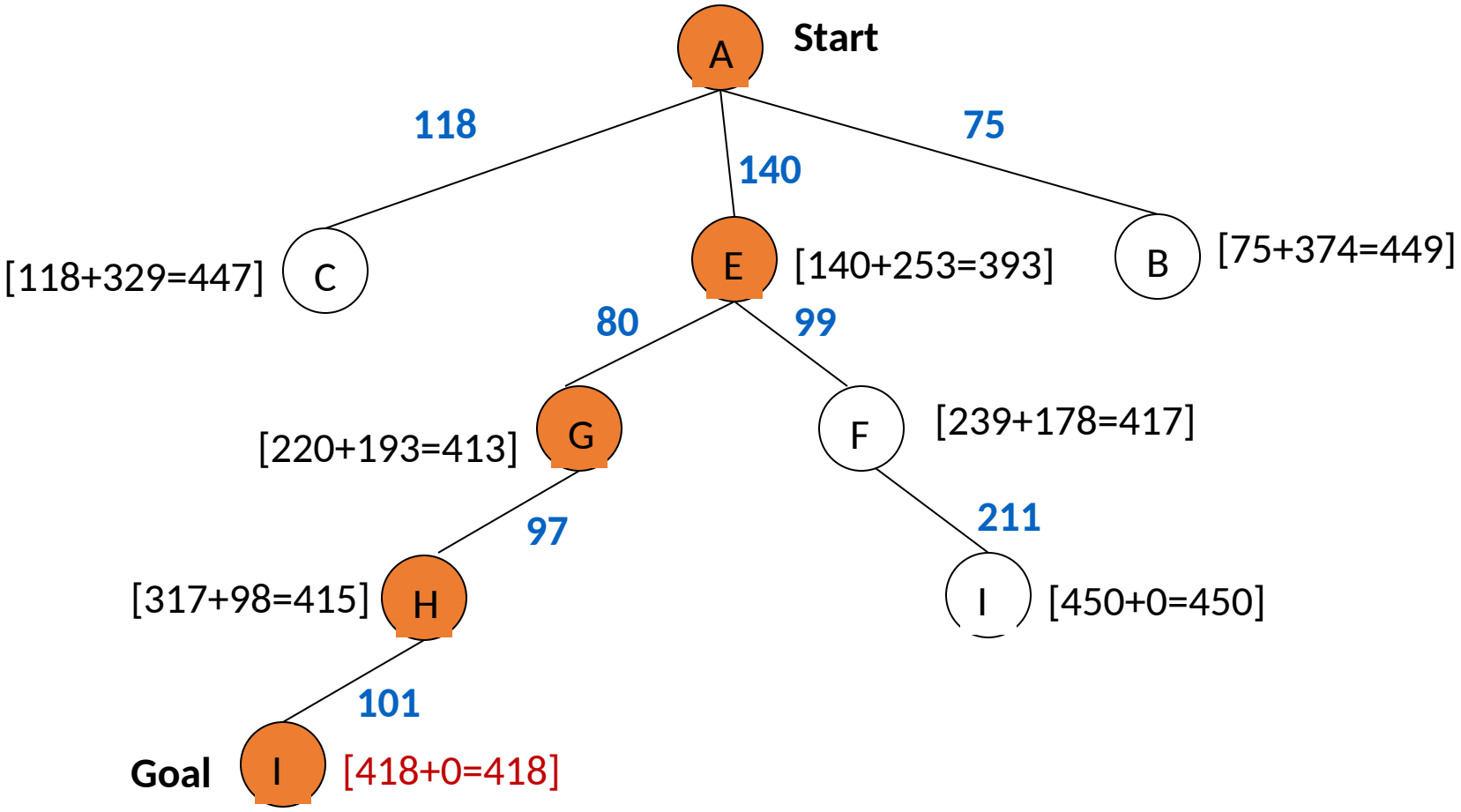
# A* Search: Tree Search



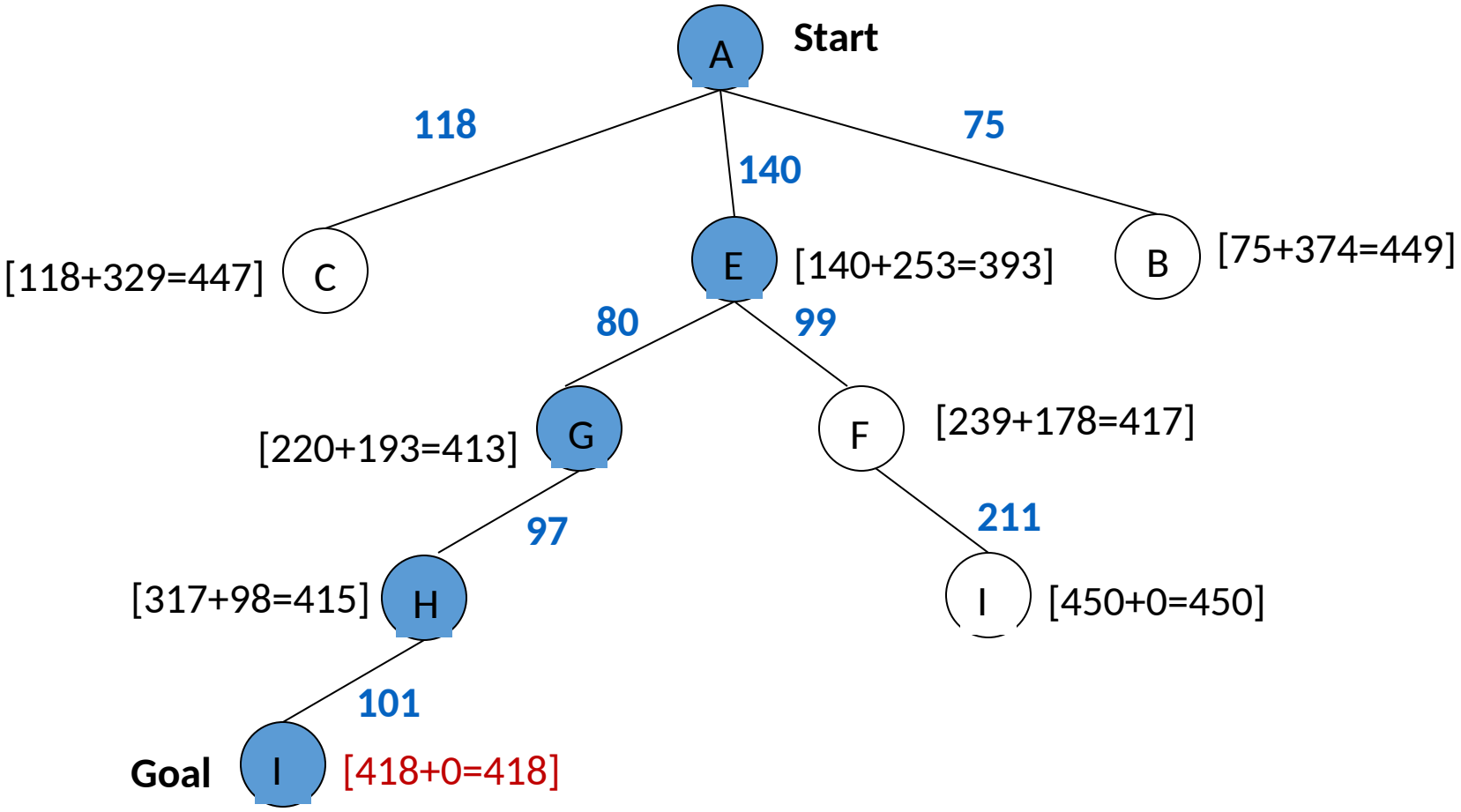
# A* Search: Tree Search



# A* Search: Tree Search

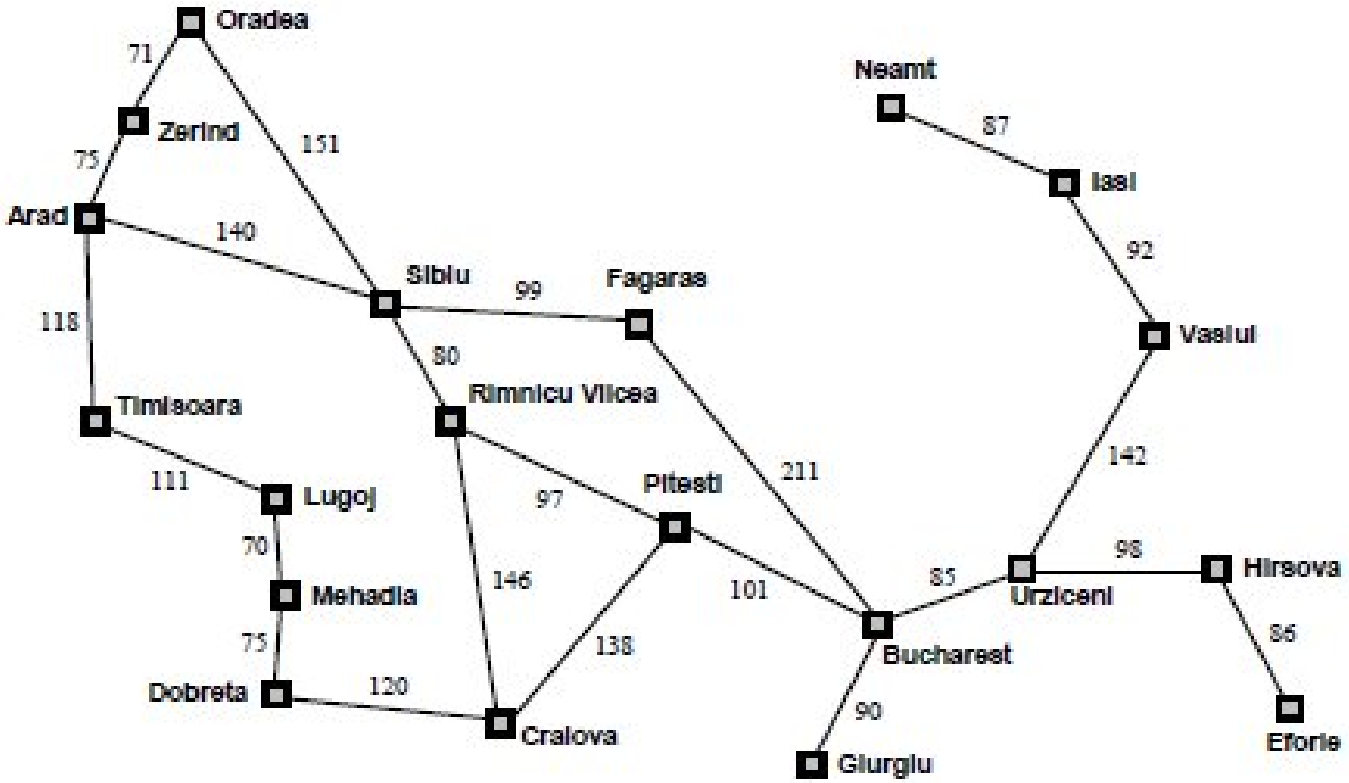


# A* Search: Tree Search




**dist(A-E-G-H-I) =140+80+97+101=418**

# A* search example

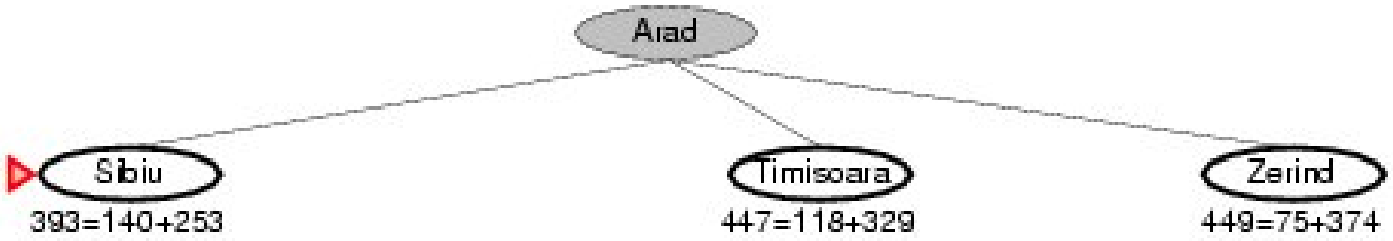



| Straight-line distance to Bucharest | |
|---|---|
| Arad | 366 |
| Bucharest | 0 |
| Craiova | 160 |
| Dobreta | 242 |
| Eforie | 161 |
| Fagaras | 178 |
| Giurgiu | 77 |
| Hirsova | 151 |
| Iasi | 226 |
| Lugoj | 244 |
| Mehadia | 241 |
| Neamt | 234 |
| Oradea | 380 |
| Pitesti | 98 |
| Rimnicu Vilcea | 193 |
| Sibiu | 253 |
| Timisoara | 329 |
| Urziceni | 80 |
| Vaslui | 199 |
| Zerind | 374 |

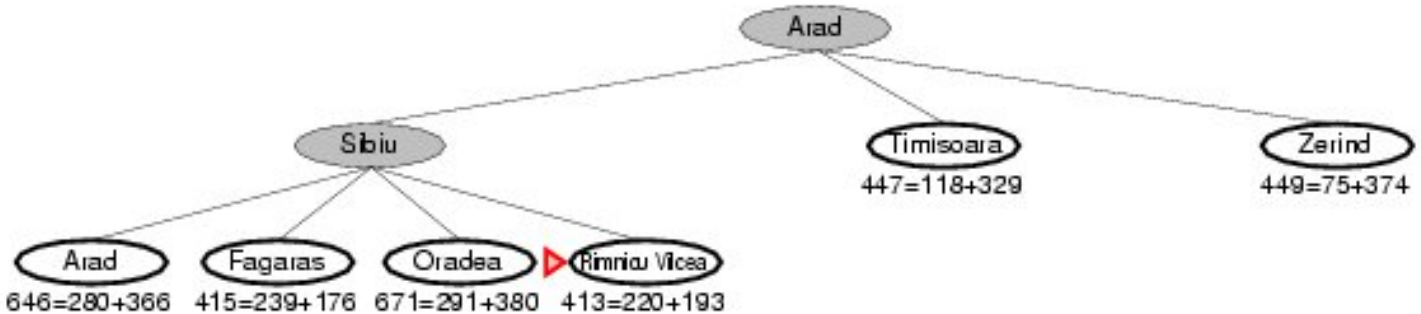
# A* search example



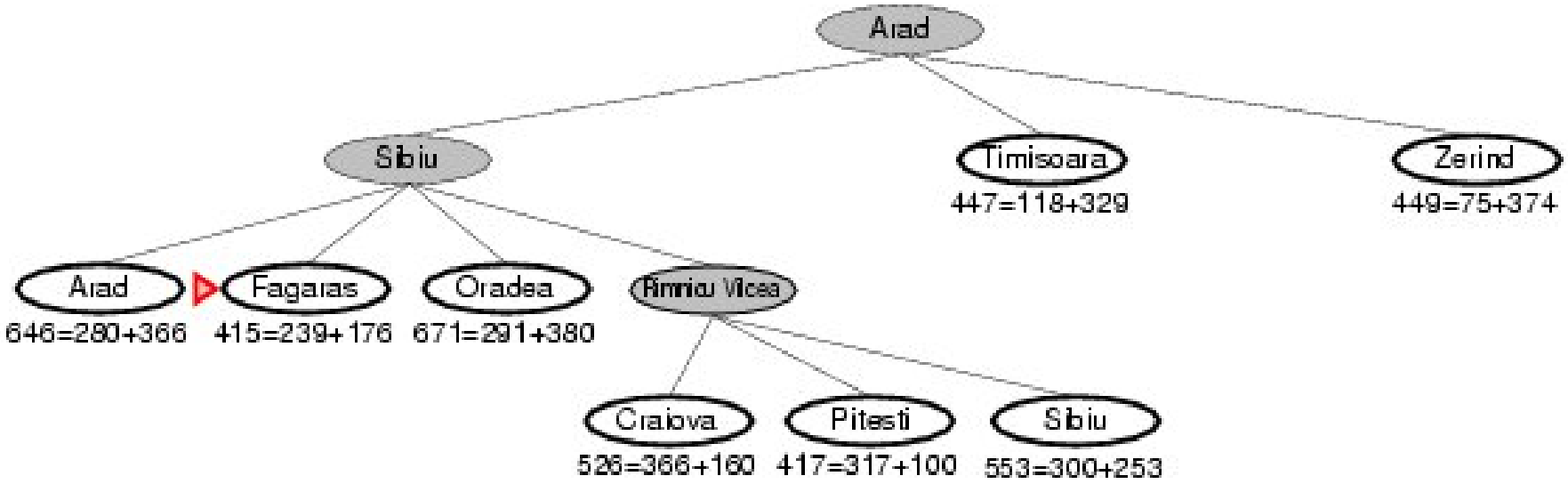
# A* search example



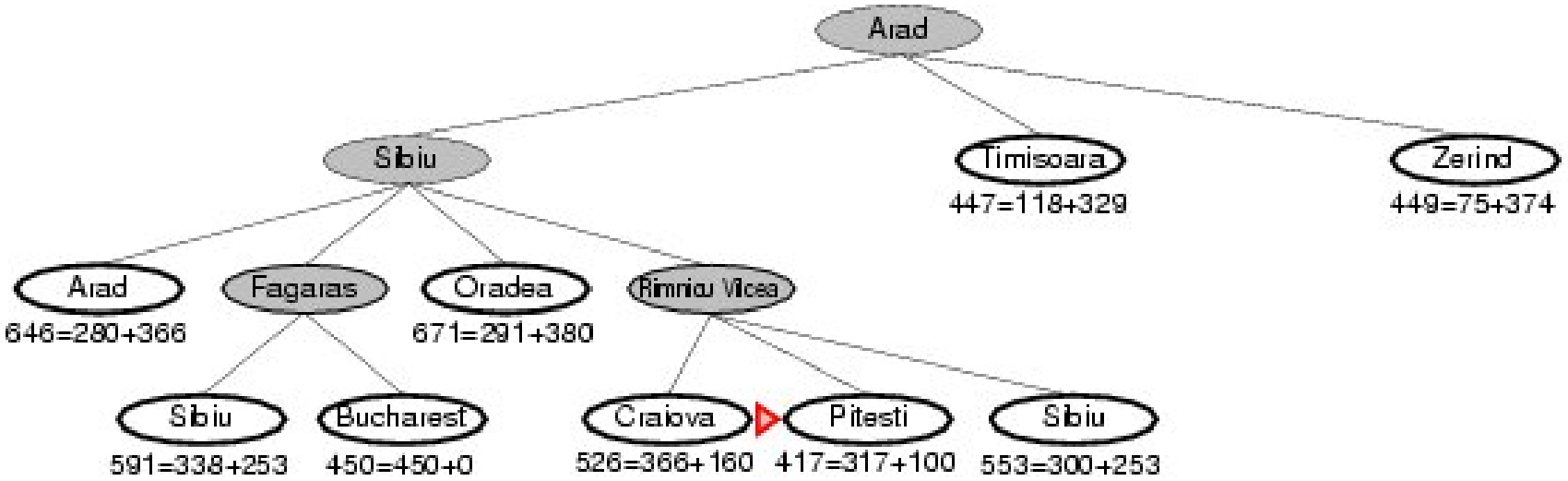
# A* search example



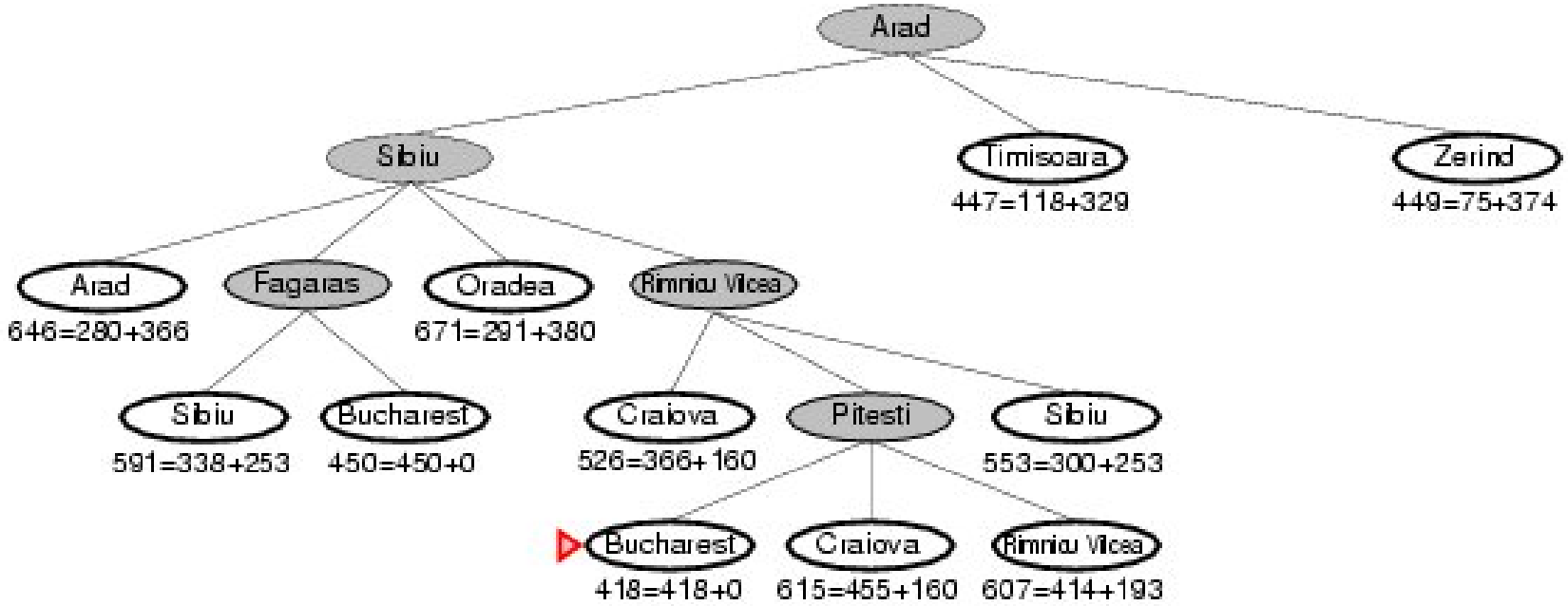
# A* search example



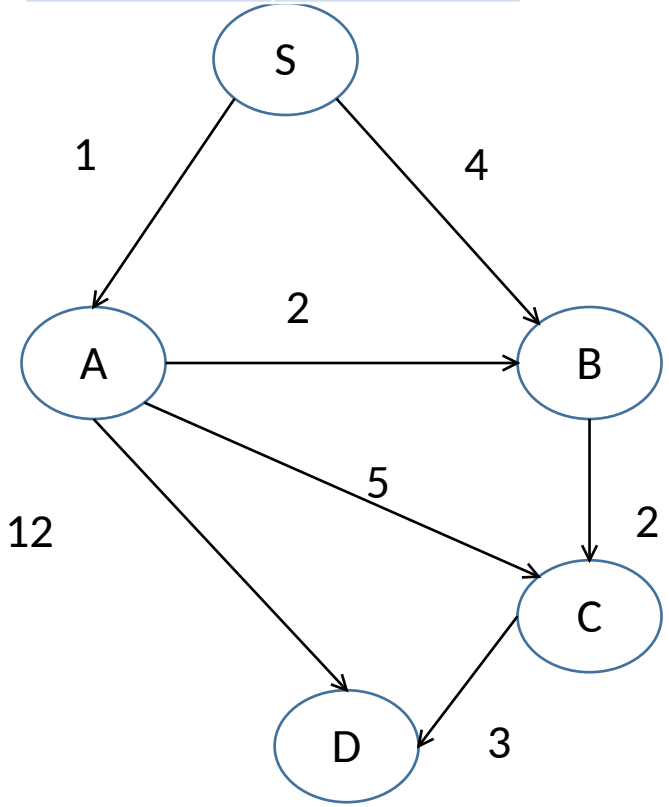
# A* search example



# A* search example



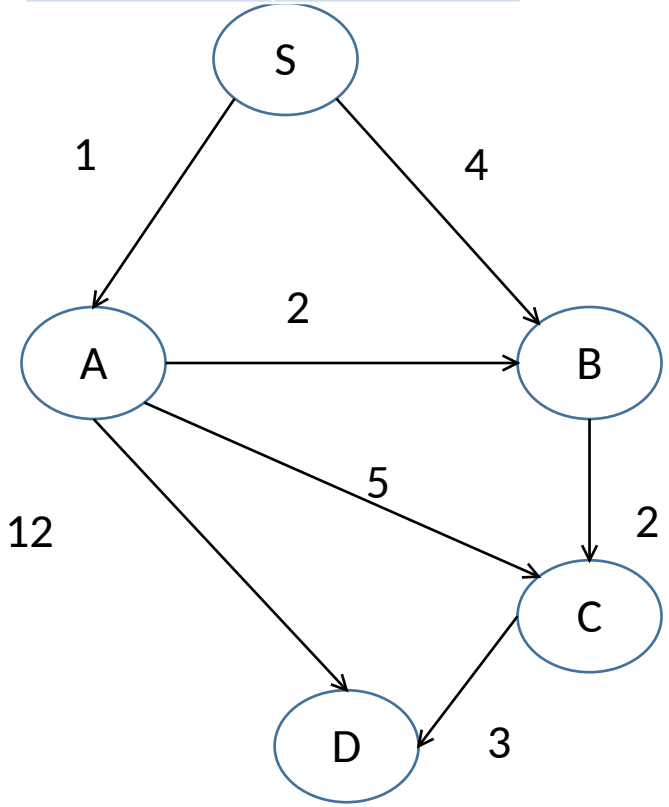
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | | Empty |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

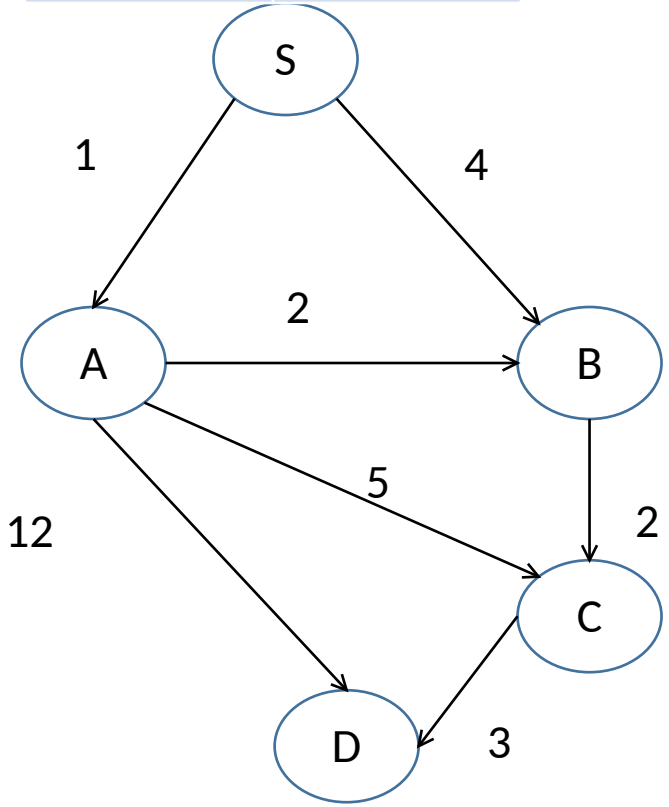
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

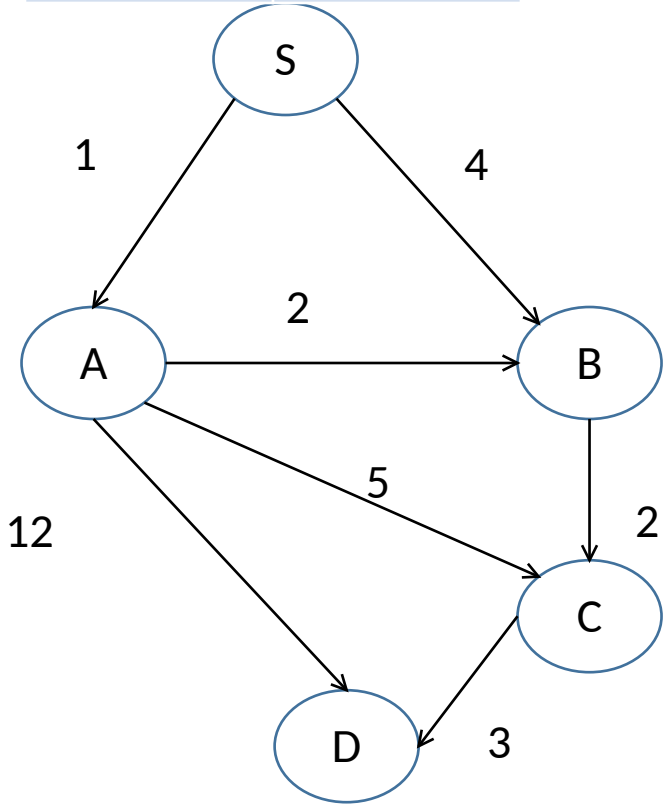
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | | S |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

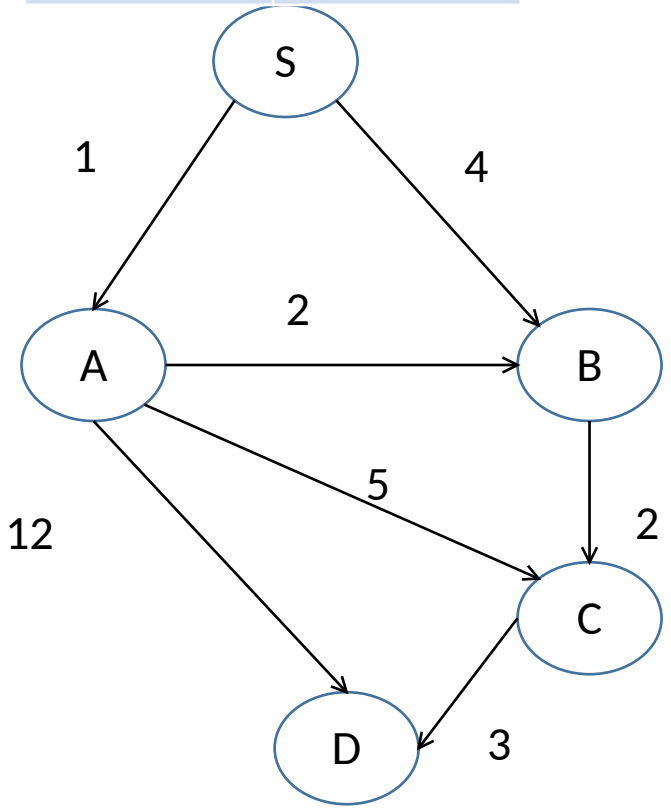
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | | | |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

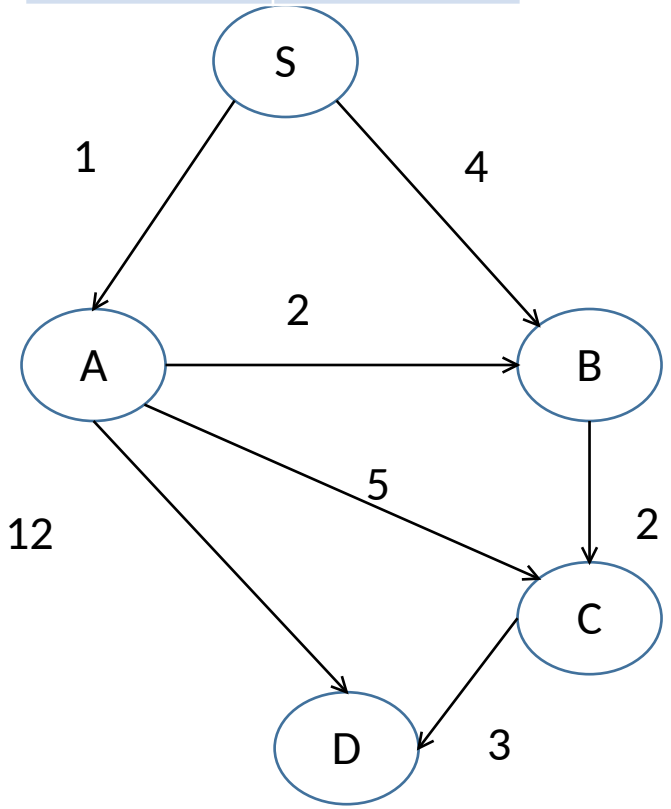
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | | S,B |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

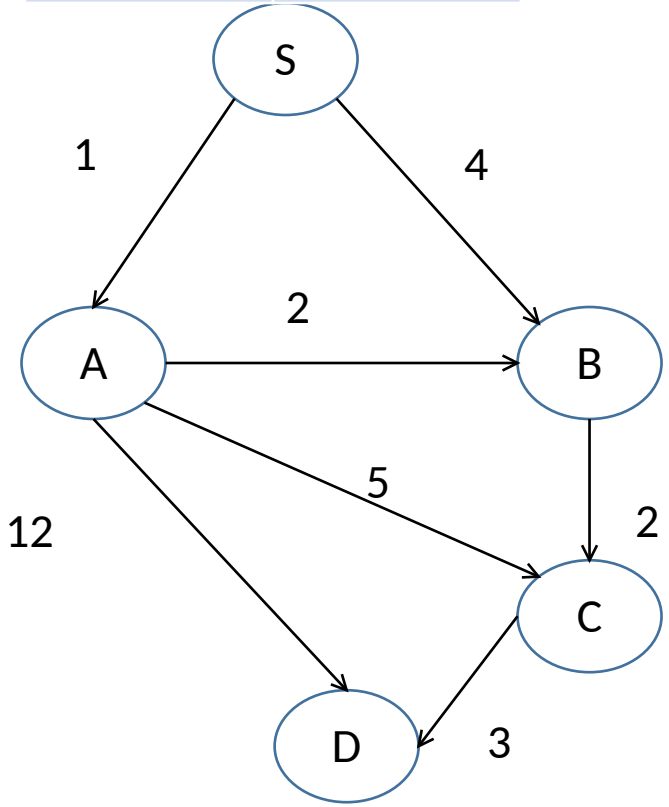
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | | | |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

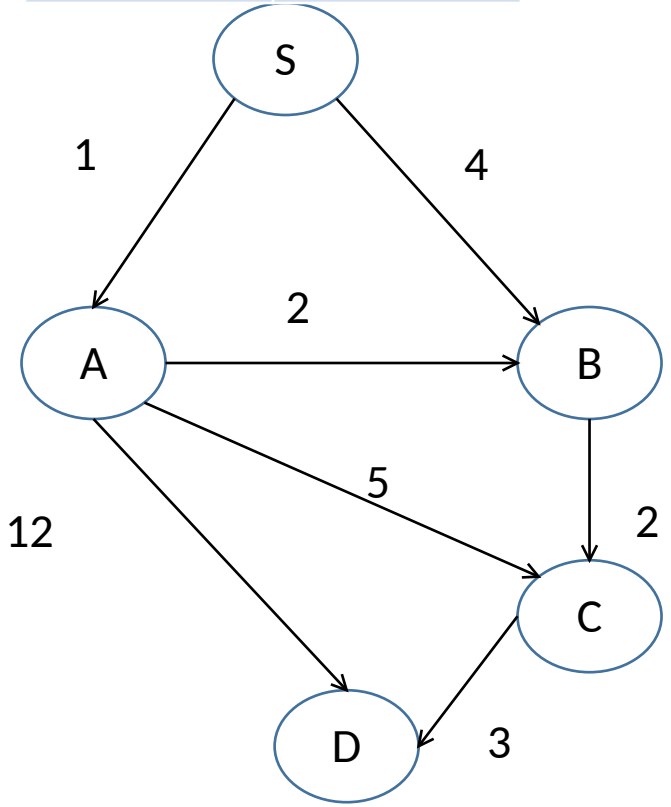
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | ~~(S-A-B,3+2=5)~~(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7) | | S,B,A |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

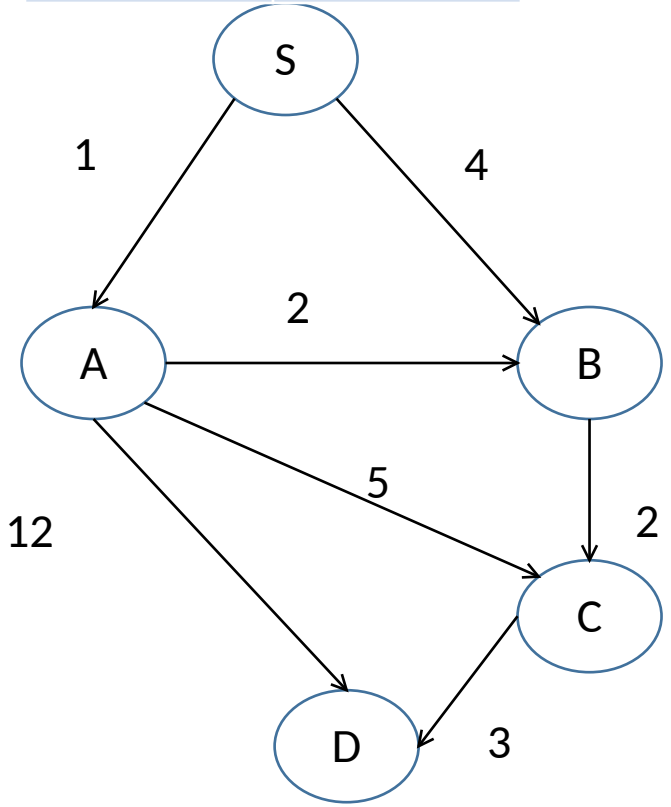
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | ~~(S-A-B,3+2=5)~~(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7) | C | S,B,A |
| 5 | | | |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

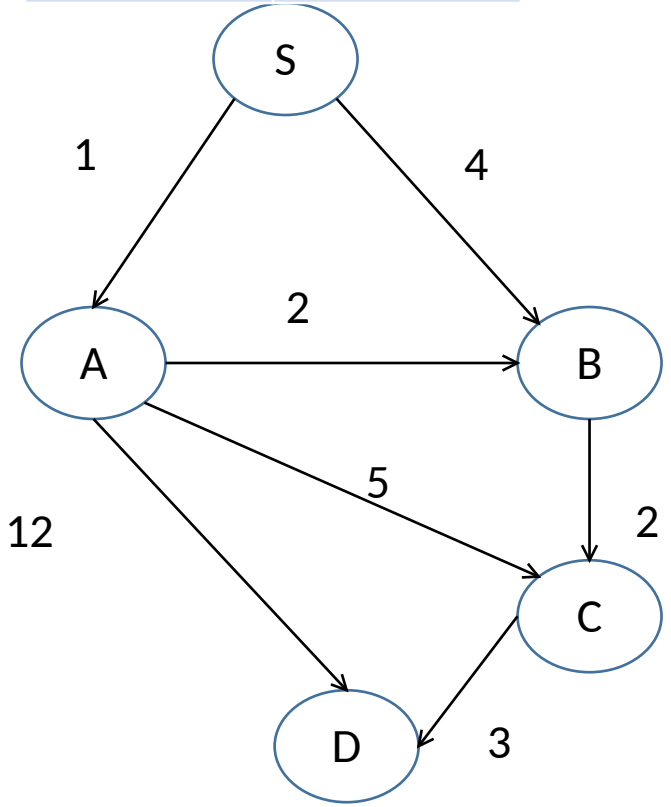
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | ~~(S-A-B,3+2=5)~~(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7) | C | S,B,A |
| 5 | (S-A-C-D,9+0=9) | | S,B,A,C |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

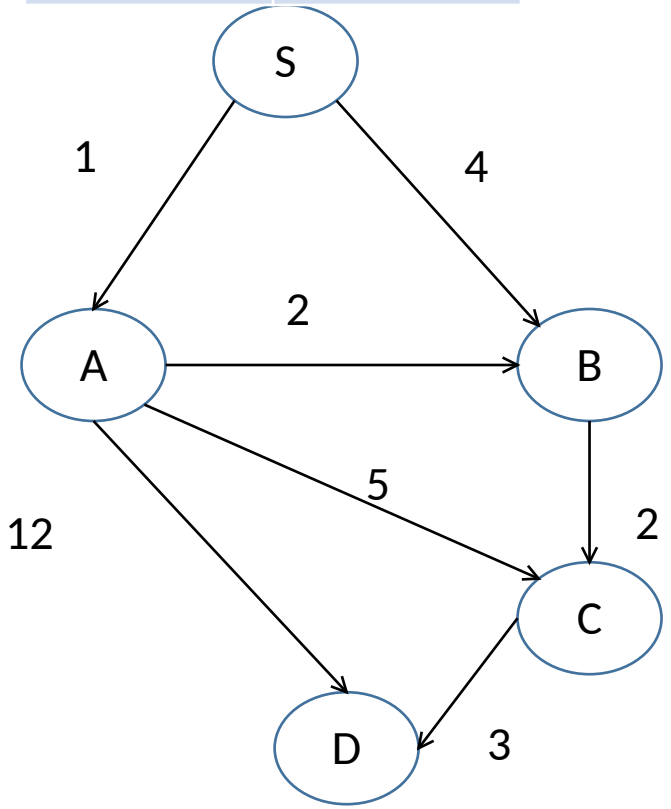
| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

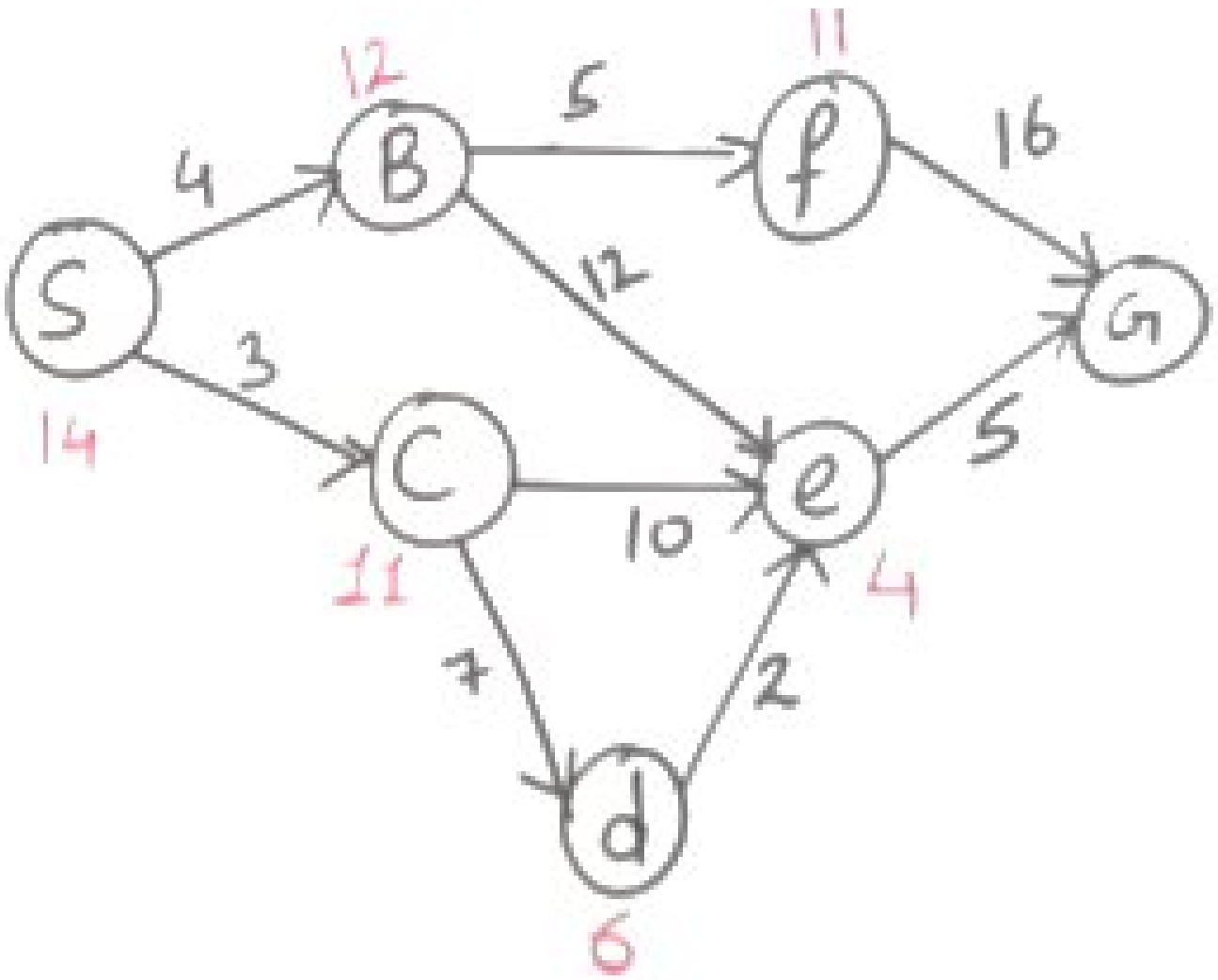
| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | ~~(S-A-B,3+2=5)~~(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7) | C | S,B,A |
| 5 | (S-A-C-D,9+0=9) | D | S,B,A,C |
| 6 | | | |
| 7 | | | |
| | | | |
| | | | |

| Heuristic Value | |
|---|---|
| S | 7 |
| A | 6 |
| B | 2 |
| C | 1 |
| D | 0 |




f(n) = g(n)+h(n)

| | FRONTIER | EXPAND | EXPLORED |
|---|---|---|---|
| 1 | (S,0+7=7) | S | Empty |
| 2 | (S-A,1+6=7)(S-B,4+2=6) | B | S |
| 3 | (S-A,7)(S-B-C,6+1=7) | A | S,B |
| 4 | ~~(S-A-B,3+2=5)~~(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7) | C | S,B,A |
| 5 | (S-A-C-D,9+0=9) | D | S,B,A,C |
| 6 | OR (S-B-C-D,9+0=9) | | |
| 7 | | | |
| | | | |
| | | | |

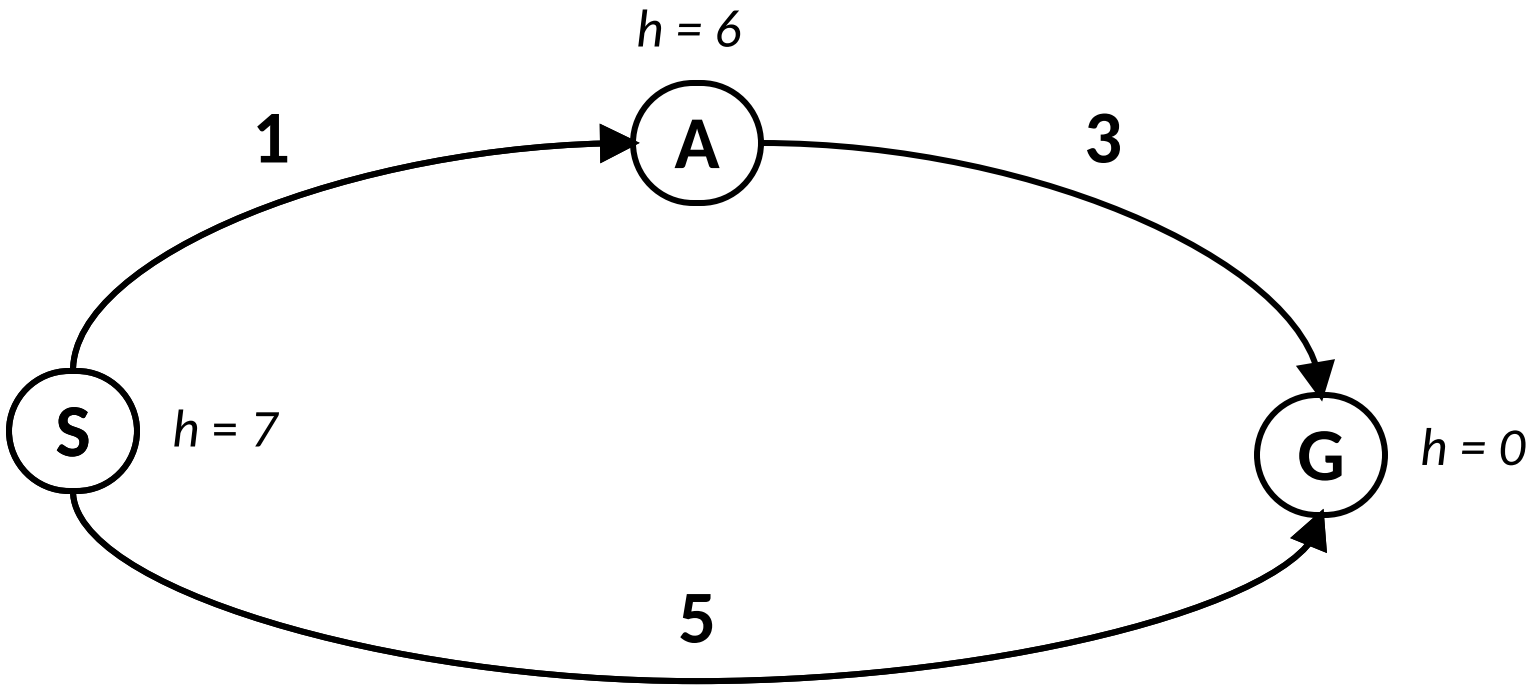



**Key:**

**Black Numbers = Edge Cost g(n)**

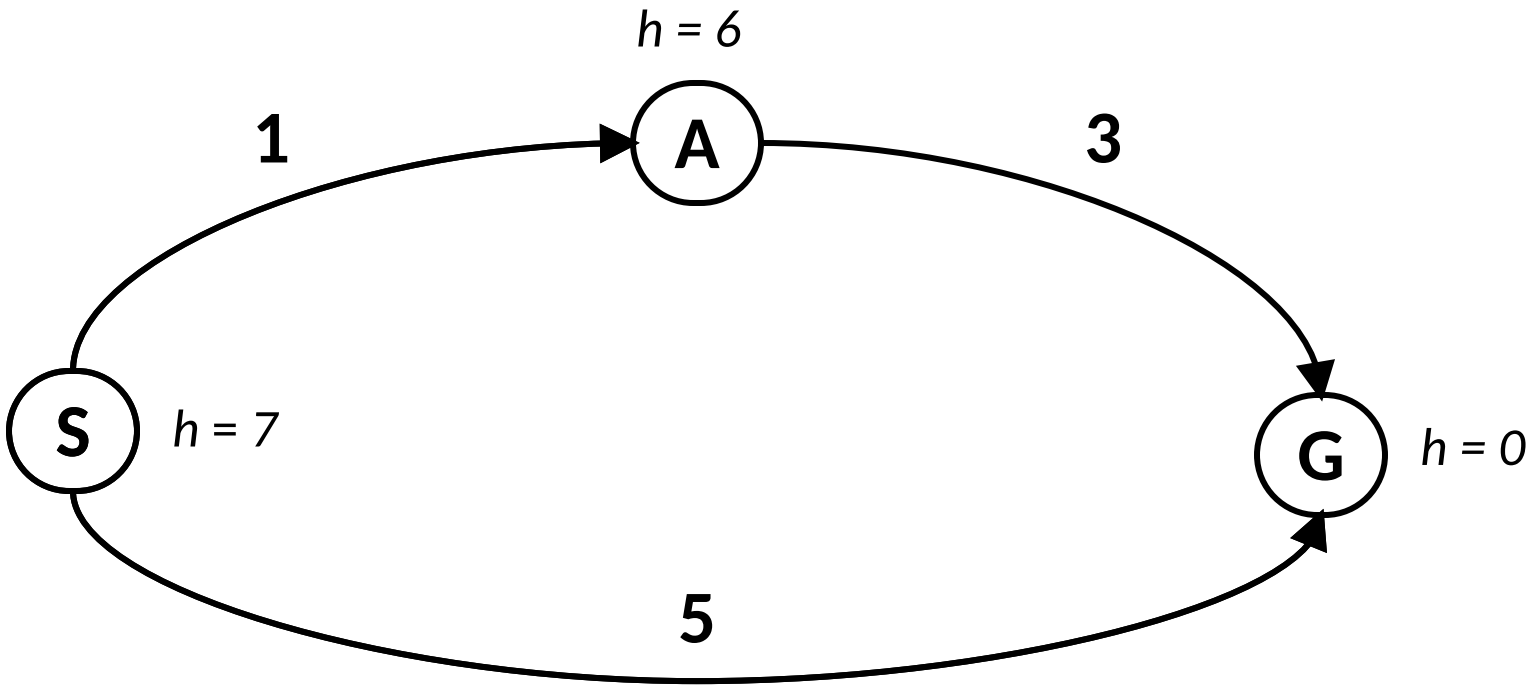
**Red Numbers = Heuristic h(n)**

# Is A* Optimal?



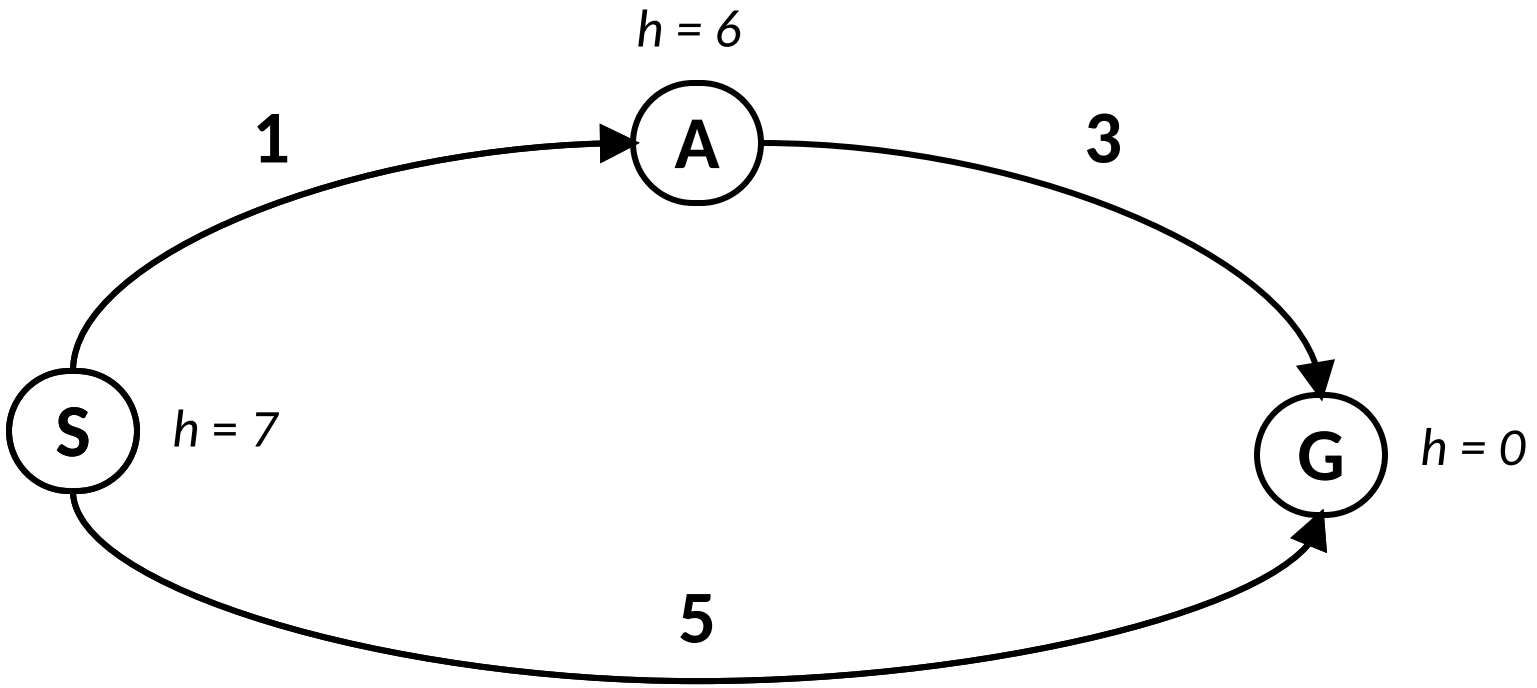

| Node | g(n) | h(n) | f(n)=g+h |
|---|---|---|---|
| S | | | |
| A | | | |
| G (via A) | | | |
| G (direct) | | | |

# Is A* Optimal?



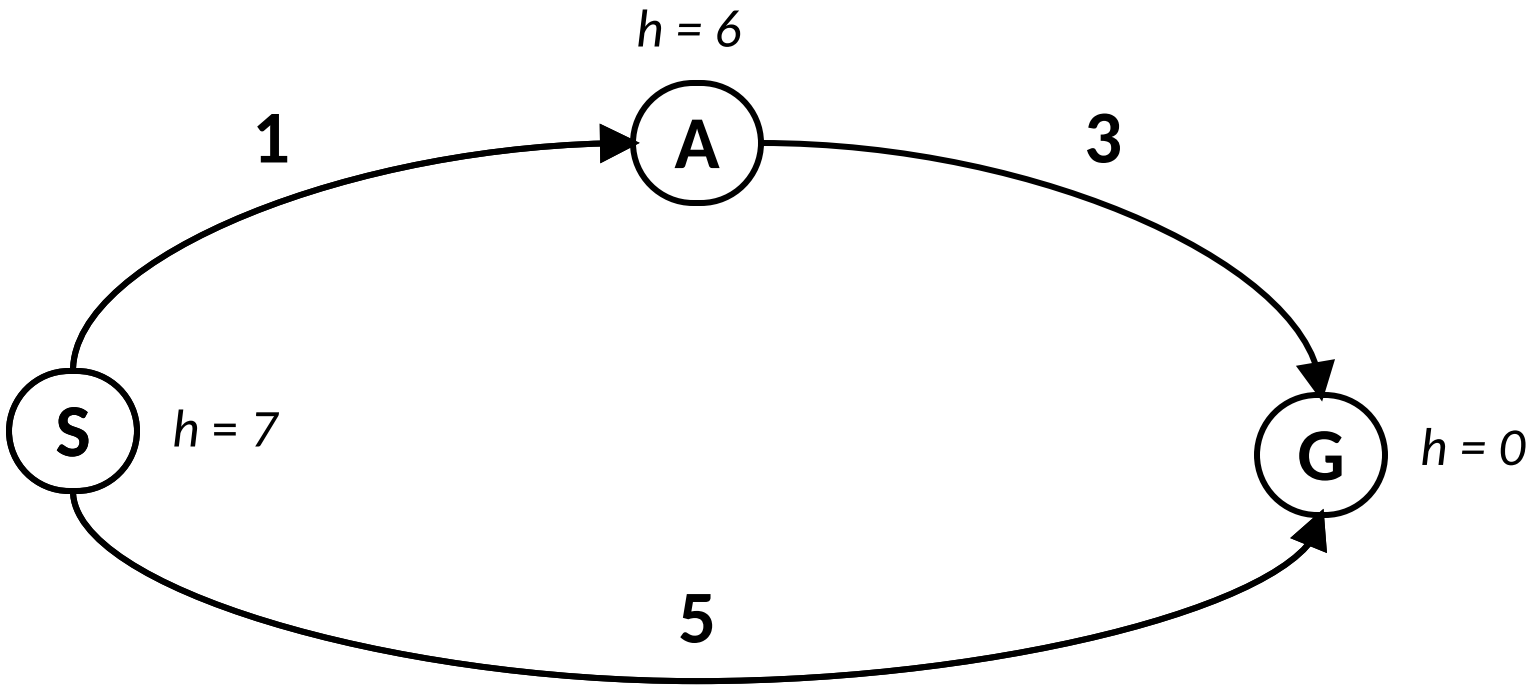

| Node | g(n) | h(n) | f(n)=g+h |
|---|---|---|---|
| S | 0 | 7 | 7 |
| A | 1 | 6 | 7 |
| G (via A) | 4 | 0 | 4 |
| G (direct) | 5 | 0 | 5 |

# Is A* Optimal?




- What went wrong?
  - Actual bad goal cost < estimated good goal cost
  - We need estimates to be less than actual costs!
  - A* is optimal **only if the heuristic never overestimates the actual remaining cost.**

# Is A* Optimal?




- A heuristic is **allowed to underestimate**. But it should **never overestimate.**

# Good Heuristic Vs Bad Heuristic

- **Good Heuristic**
  - Estimate ≤ Actual Remaining Cost
- **Bad Heuristic**
  - Estimate > Actual Remaining Cost

- Once the heuristic starts overestimating, A* can lose its guarantee of optimality.

# Admissible Heuristic Functior

- A heuristic $h(n)$ is admissible if $h(n)$ never overestimates the cost to reach the goal.
- Mathematically $h$ is admissible if

  For all $n$, $h(n) \leq C(n)$

  Where
  - $n$ is the node, $h$ is heuristic
  - $h(n)$ is the cost indicated by $h$ to reach goal from $n$
  - $C(n)$ is the actual cost to reach a goal from $n$



# Admissible heuristics Examples

- Example 1: Straight-Line Distance (SLD)
  - Heuristic: Straight-line distance from the current city to the destination
  - The straight-line distance is always **less than or equal to** the actual road distance.
  - Therefore,

$$hSLD(n) \leq h\ (n)$$

- Example 2: Misplaced Tiles (8-Puzzle)

Cu e:

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | □ | 8 |

C

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | □ |

Misplaced tiles = **1**

- The heuristic counts only the misplaced tiles.
- Each misplaced tile requires **at least one move**.
- Therefore, the heuristic never overestimates the true solution cost.

✅ **Admissible Heuristic**

# Why is A* Optimal with an Admissible Heuristic?

- **Step 1: Actual Cost** $g(n)$
  - Exact cost from the Start node to the current node.
  - **Always correct** (never estimated).
- **Step 2: Estimated Remaining Cost** $h(n)$
  - Estimated cost from the current node to the Goal.
  - If **h(n)** is admissible, it **never overestimates**.,

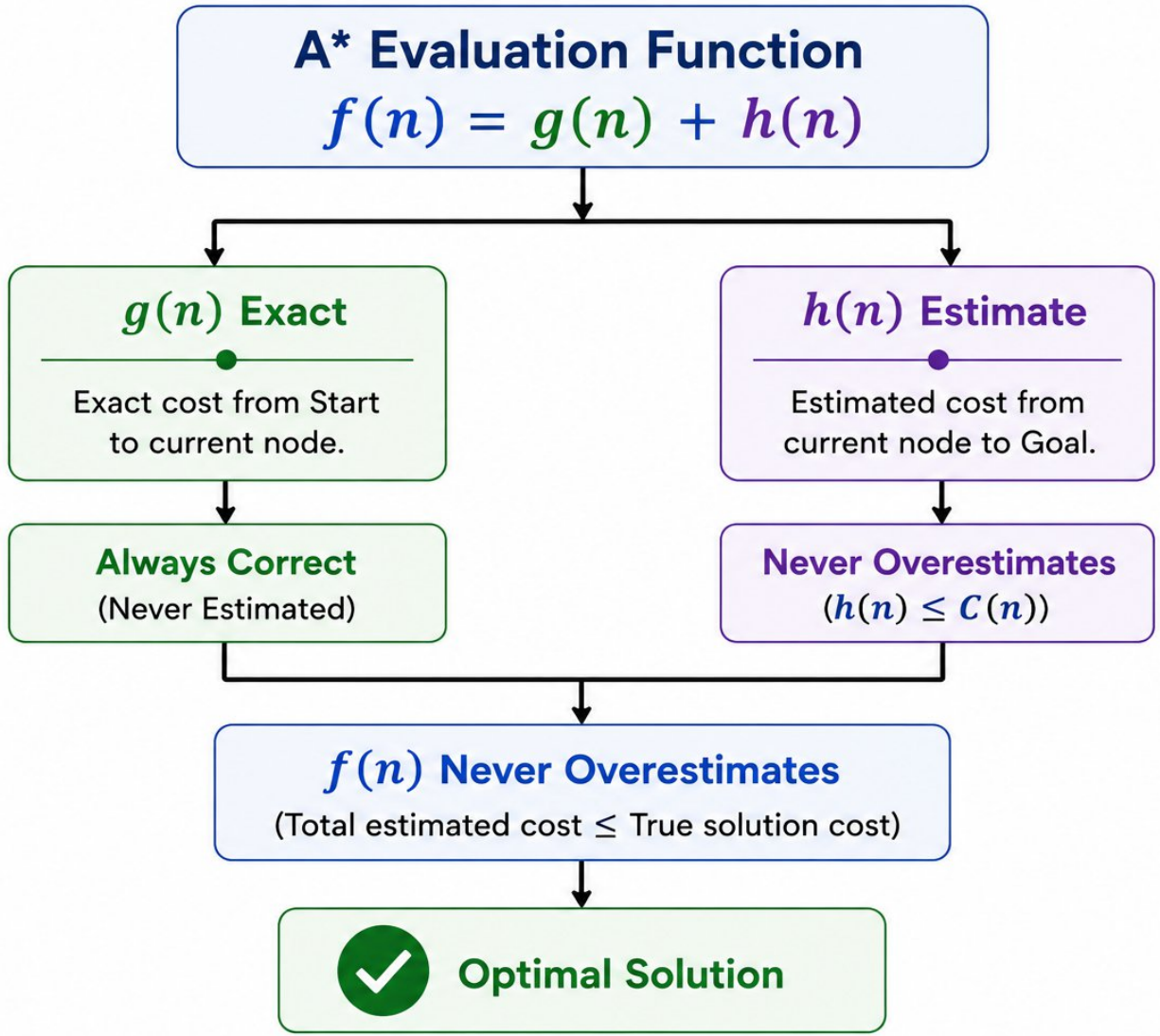
$$h(n) \leq C(n)$$

- **Step 3: Evaluation Function** $f(n) = g(n) + h(n)$

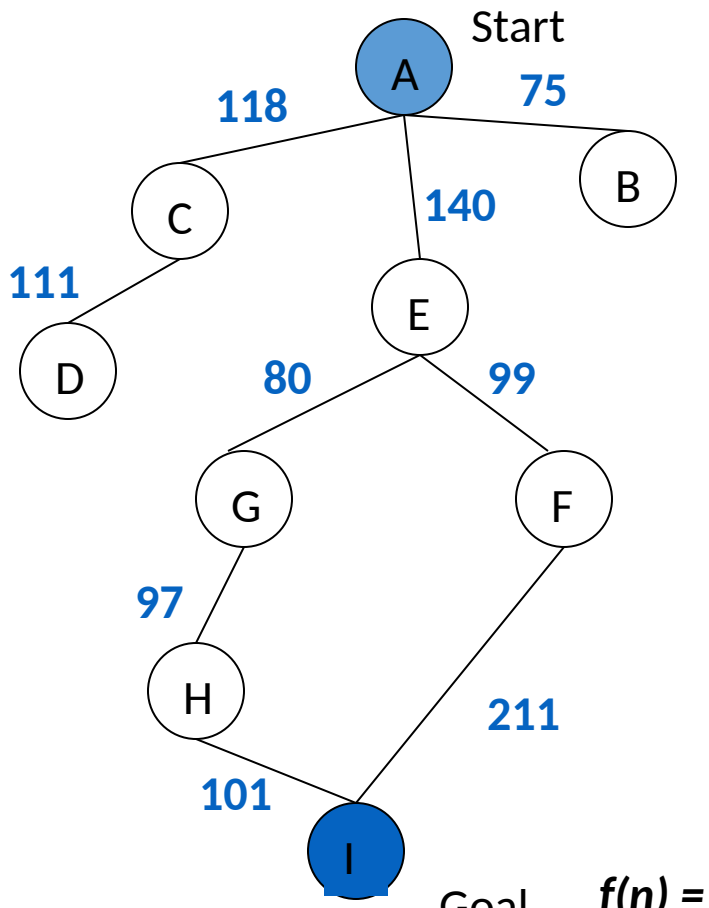
Since

$g(n)$ is exact and $h(n)$ never overestimates

*If the heuristic is admissible*, A Tree Search is guaranteed to find the optimal solution.


A* Evaluation Function
$f(n) = g(n) + h(n)$
$g(n)$ Exact
Exact cost from Start to current node.
Always Correct
(Never Estimated)
$h(n)$ Estimate
Estimated cost from current node to Goal.
Never Overestimates
($h(n) \leq C(n)$)
$f(n)$ Never Overestimates
(Total estimated cost ≤ True solution cost)
Optimal Solution

# A* Search: if h not admissible




| State | Heuristic: h(n) |
|---|---|
| A | 366 |
| B | 374 |
| C | 329 |
| D | 244 |
| E | 253 |
| F | 178 |
| G | 193 |
| **H** | **138** |
| I | 0 |

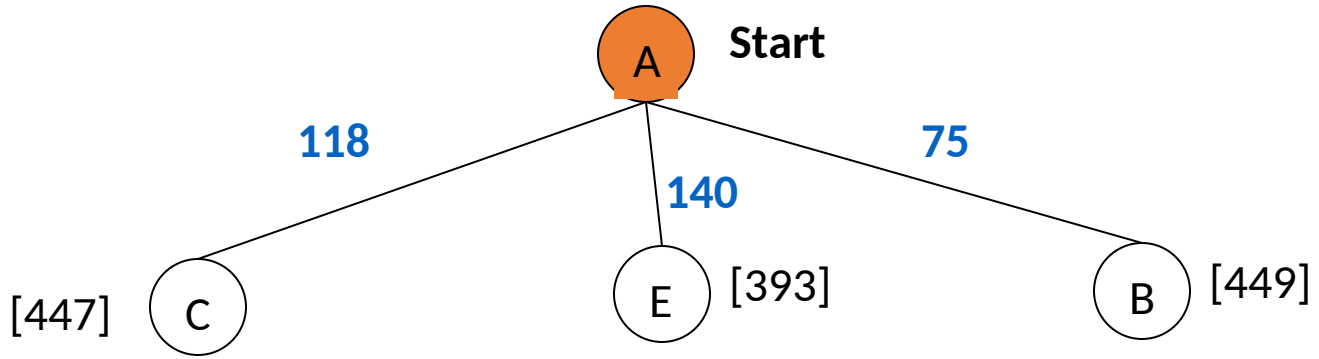
***f(n) = g(n) + h (n)*** **– (H-I) Overestimated**

**g(n):** is the exact cost to reach node *n* from the initial state.

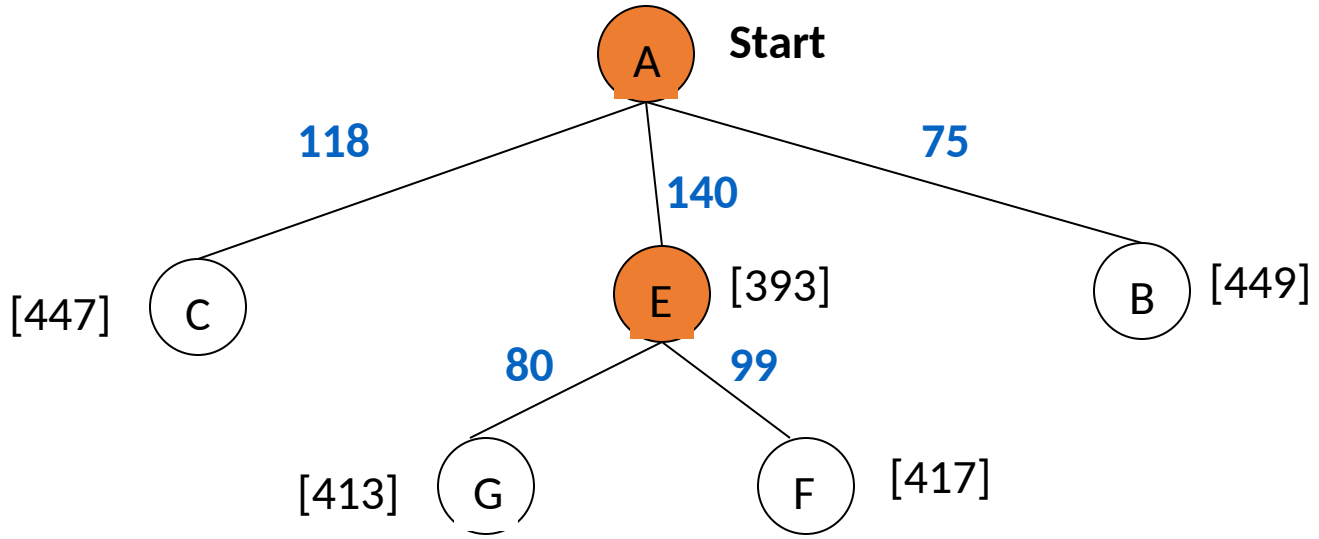
# A* Search: Tree Search



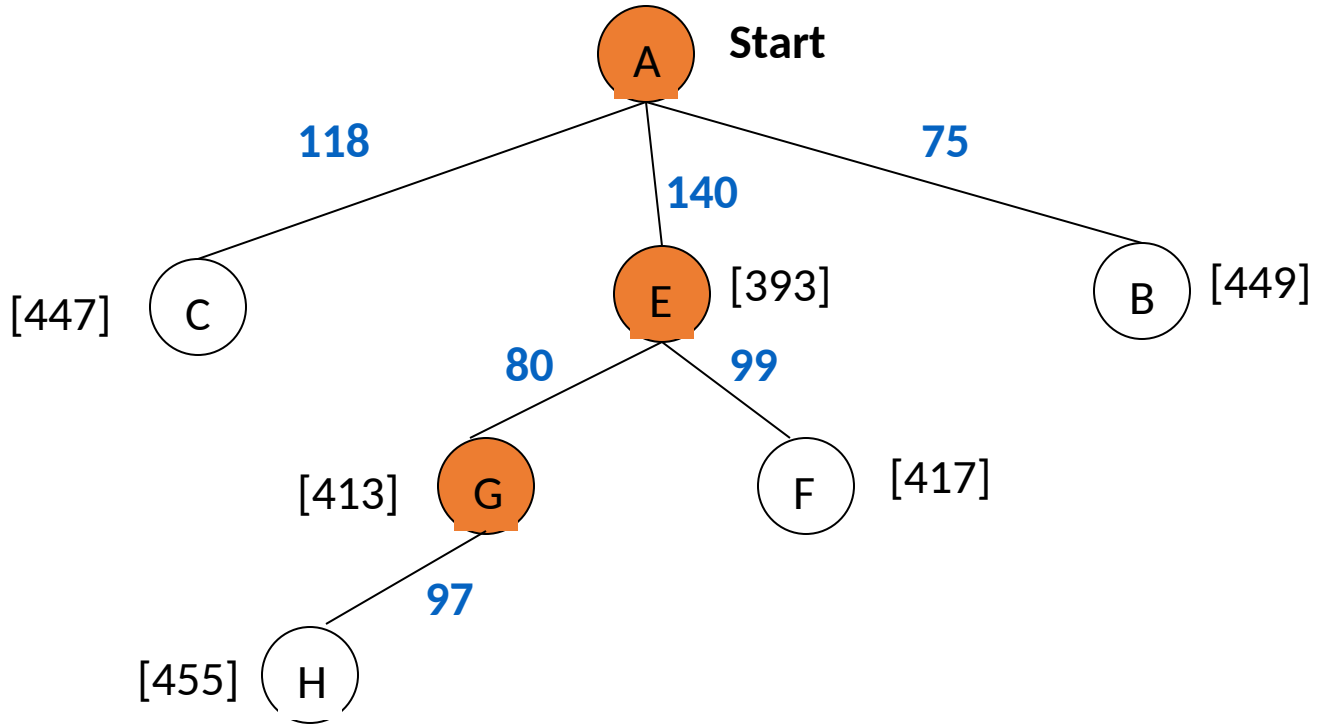
# A* Search: Tree Search



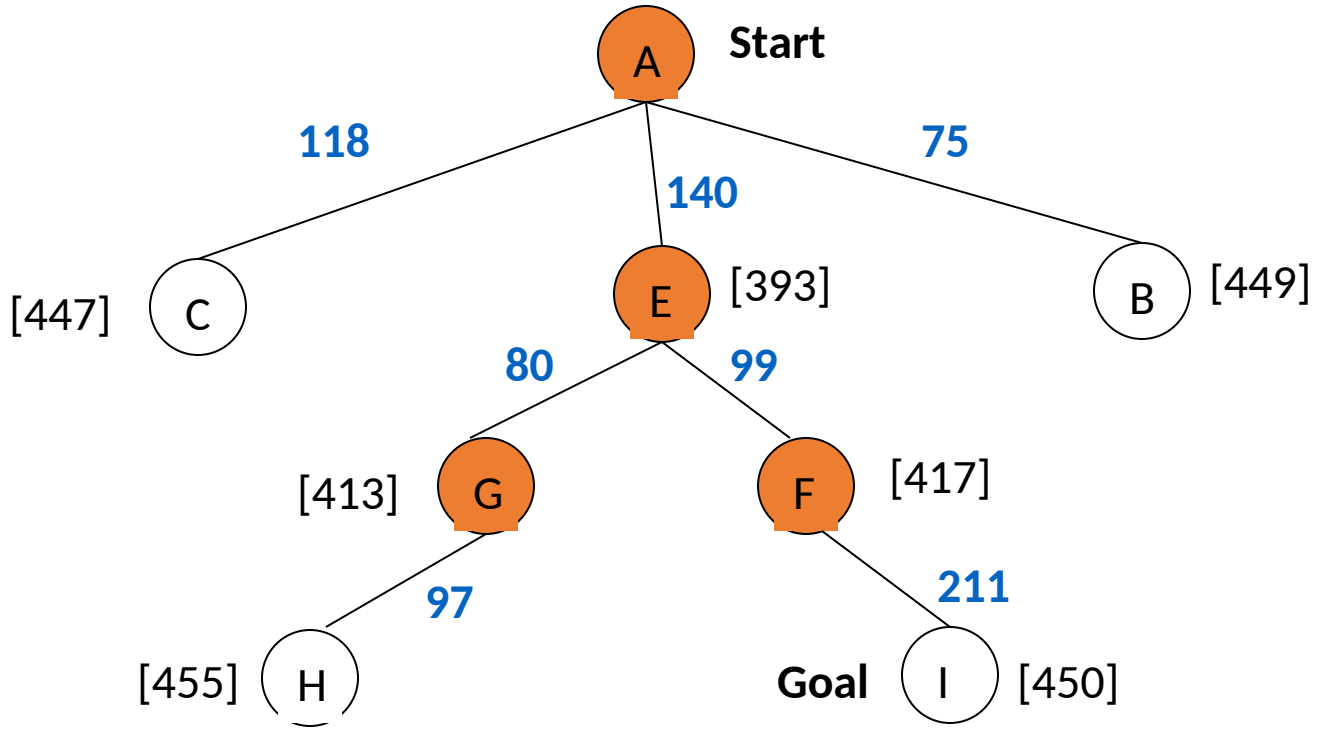
# A* Search: Tree Search



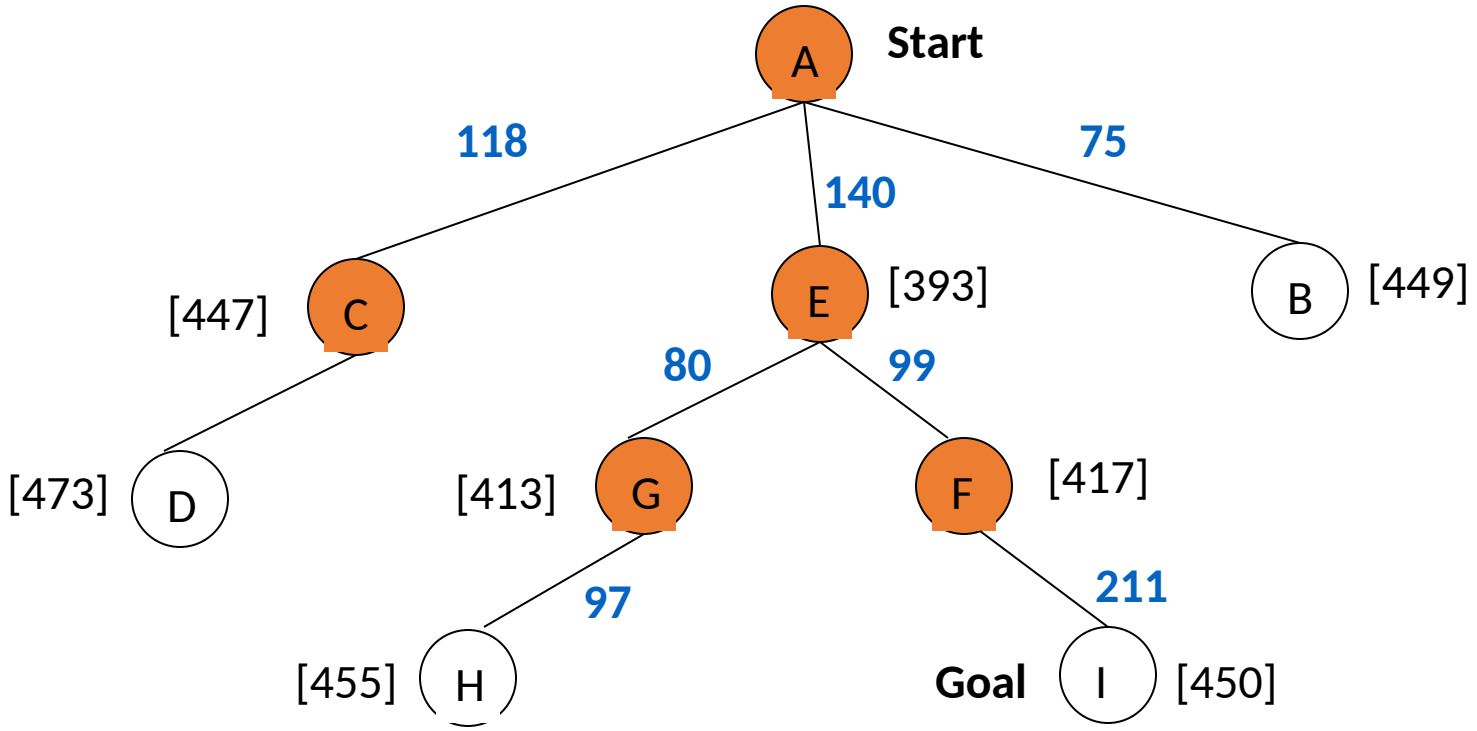
# A* Search: Tree Search



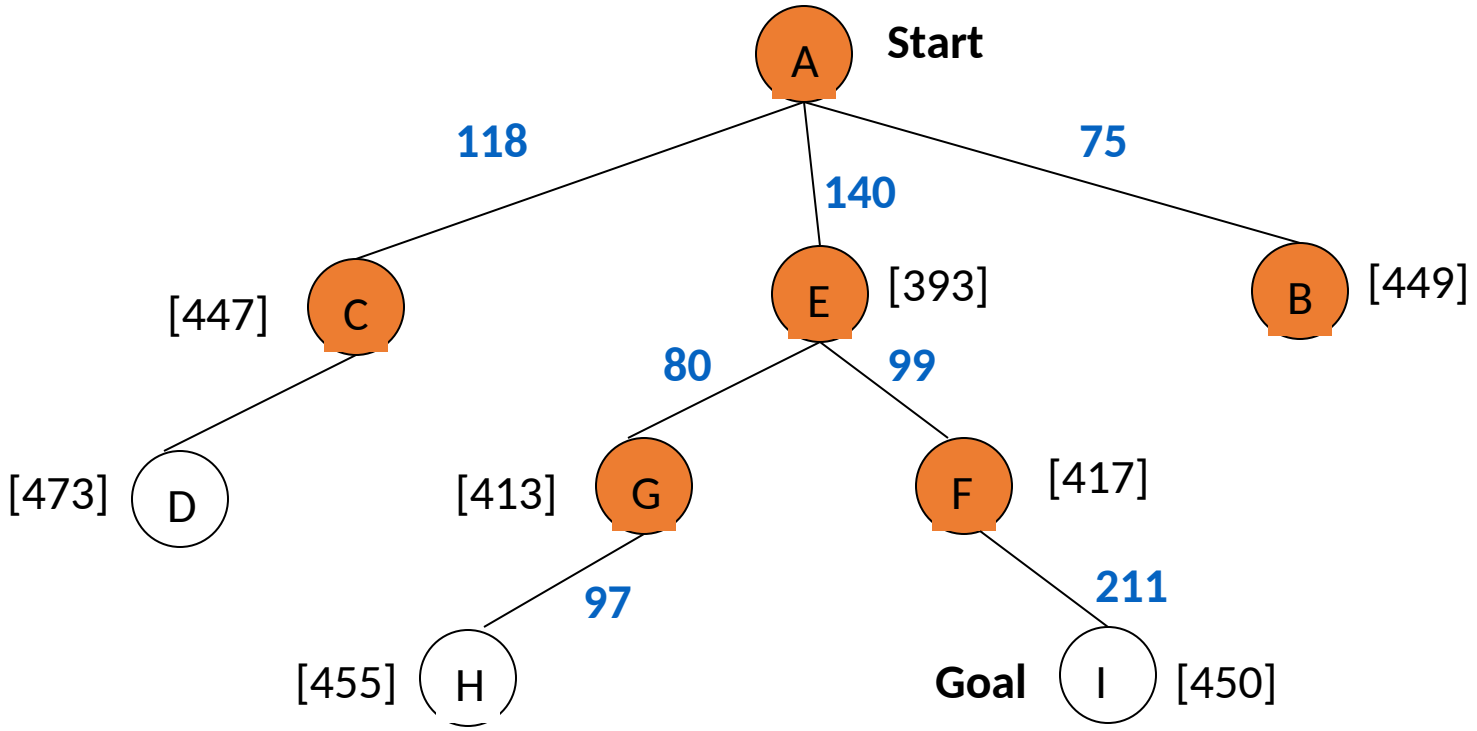
# A* Search: Tree Search



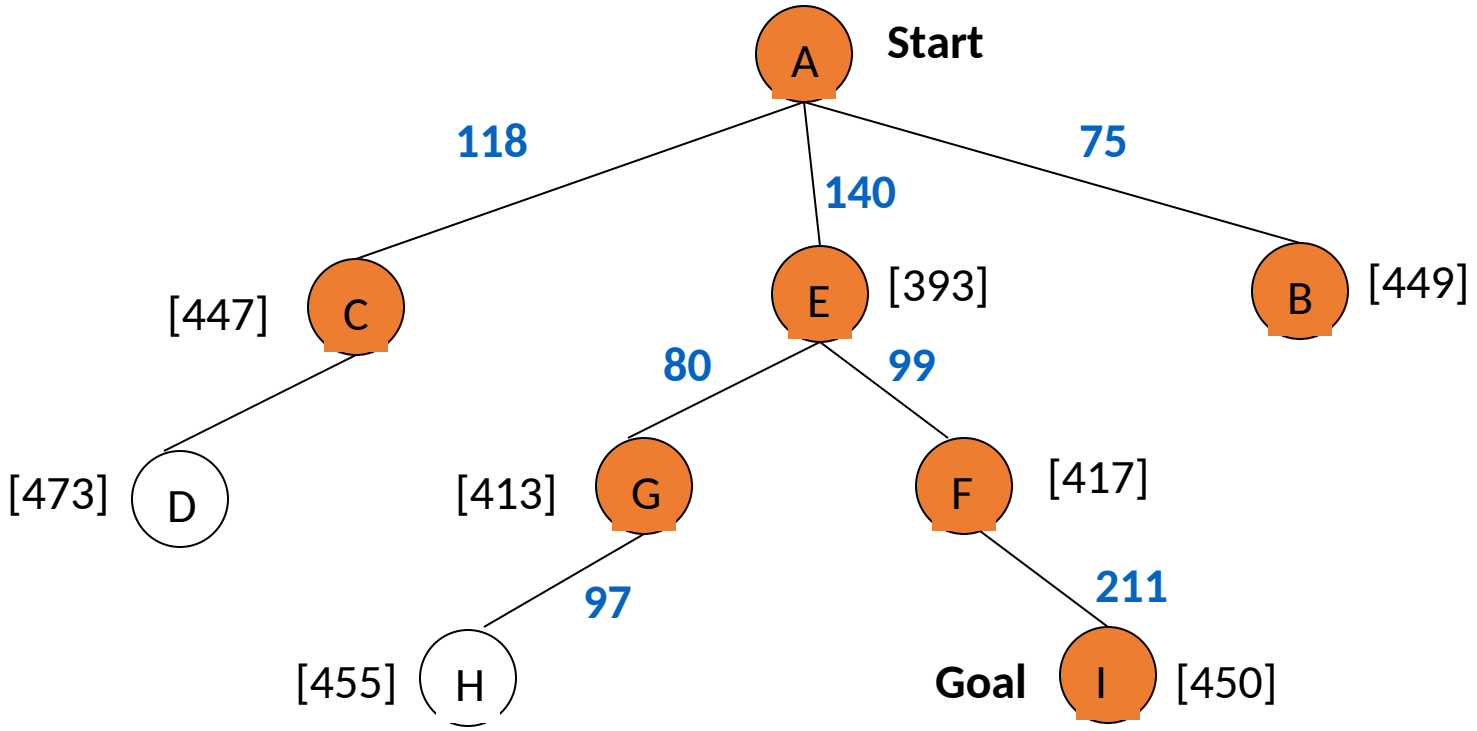
# A* Search: Tree Search



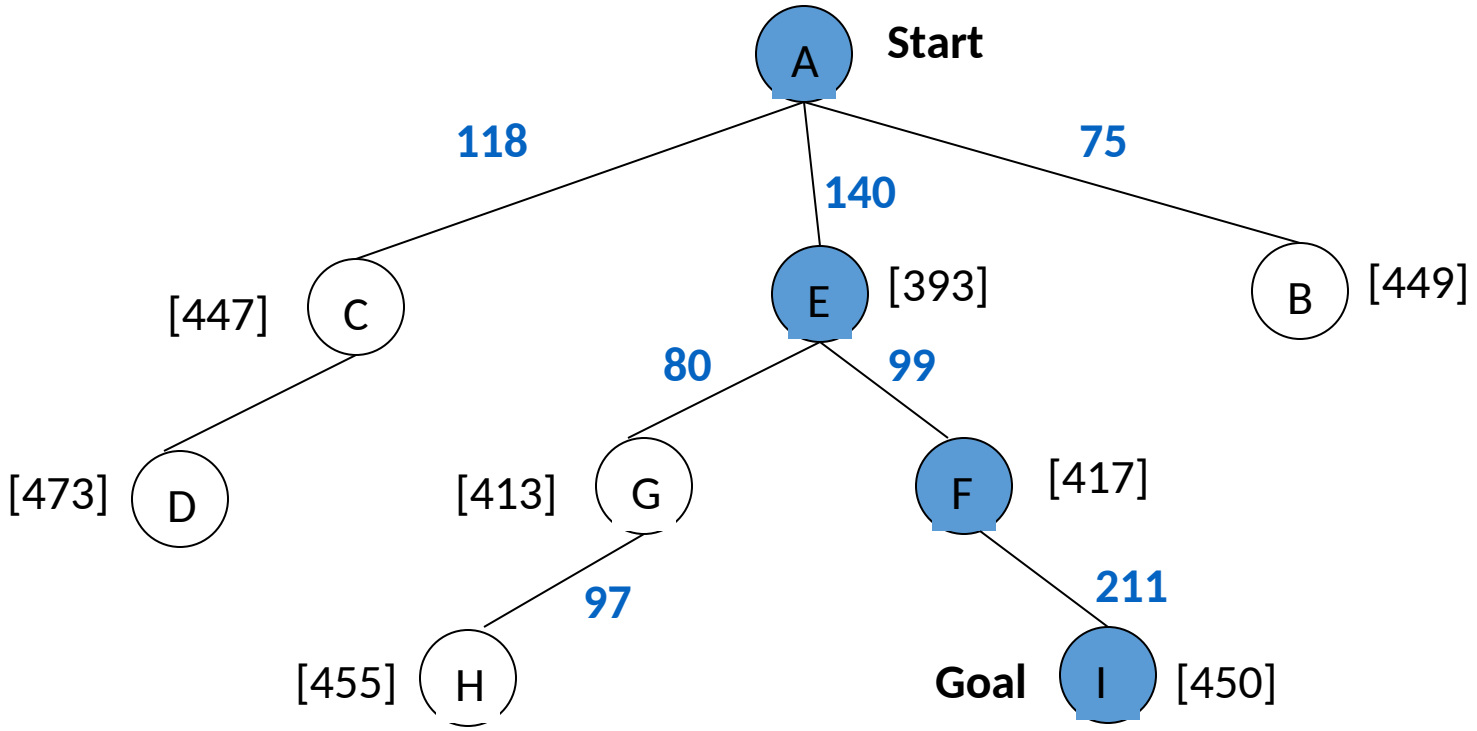
# A* Search: Tree Search



# A* Search: Tree Search



# A* Search: Tree Search



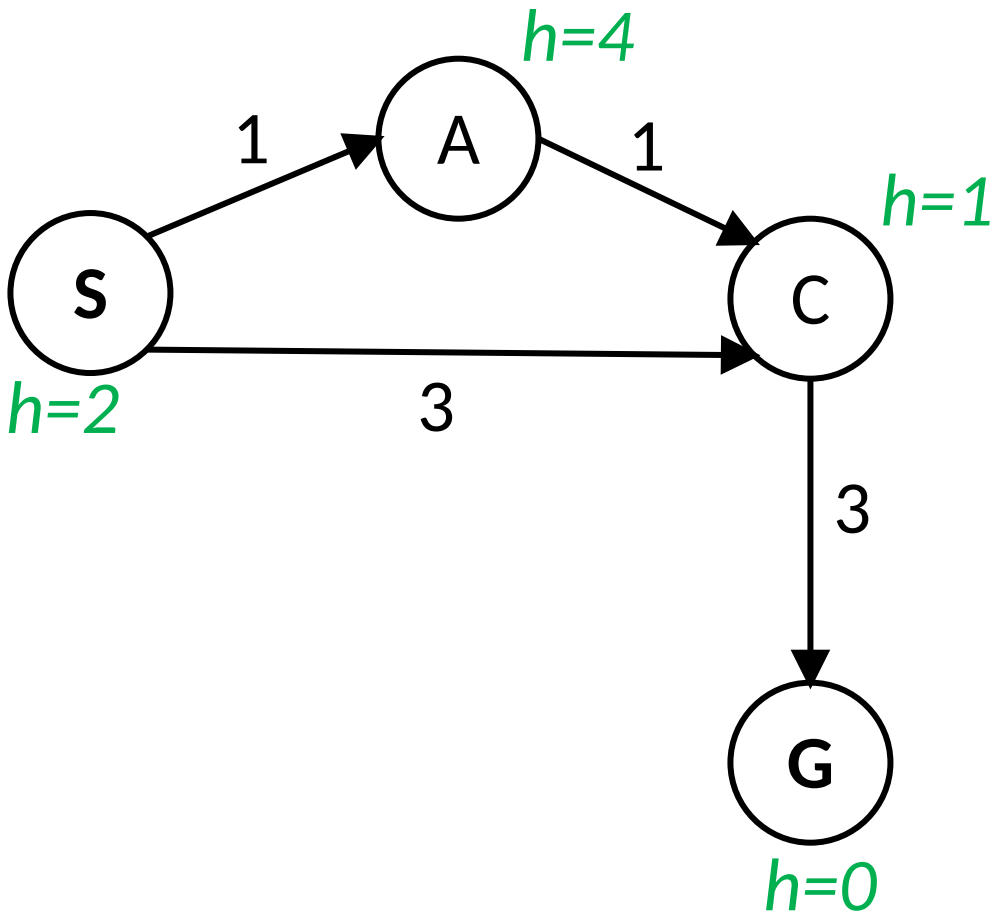

A* not optimal !!!

When heuristic h is not admissible

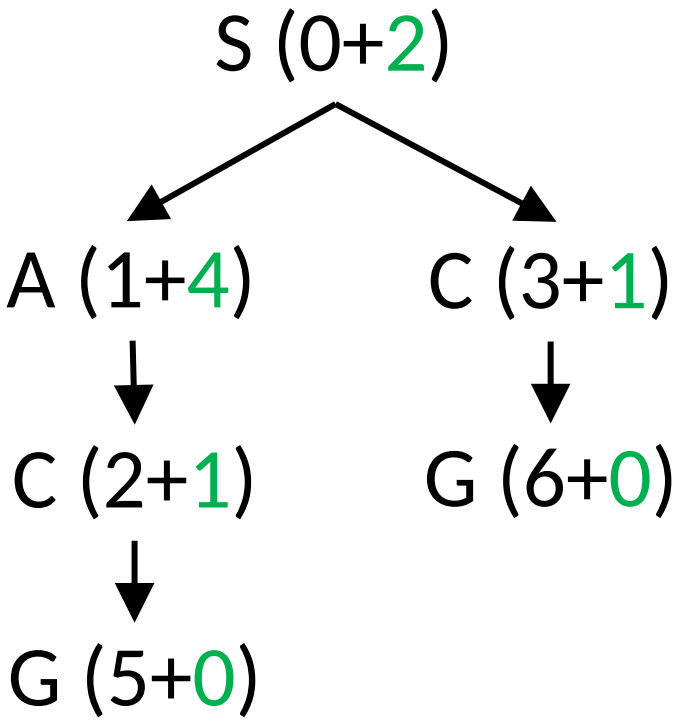
**dist(A-E-F-I) =140+99+211=450**

# A* Tree Search

State space graph




Search tree



# Consistency (Monotonicity) of Heuristics

$h(n)$: heuristic estimate from $n$ to goal

$c(A,C)$: actual cost to move from A to C

**Goal** node: G

**Main Idea:** A heuristic is **consistent** if moving from one node to a neighbor **never reduces** the estimated remaining cost by more than the **actual step cost**.

## Consistency Condition

For every edge (A, C):

$$h(A) \leq c(A,C) + h(C)$$

Equivalent form:

$$h(A) - h(C) \leq c(A,C)$$

This is called the **Triangle Inequality** for heuristics.

## Intuition (Think Google Maps)

Your estimate to the goal should decrease **gradually**, **not suddenly**!

**Consistent**

Lahore to Islamabad = 270 km

↓

Drive 30 km to Gujranwala

↓

Remaining to Islamabad ≈ 240 km

(Decrease is reasonable)

**Inconsistent**

Lahore to Islamabad = 270 km

↓

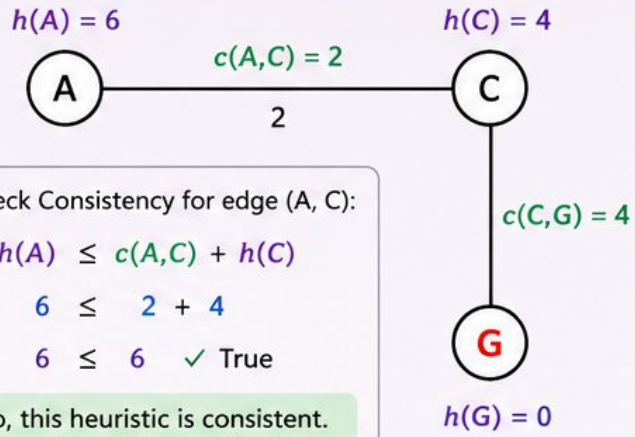
Drive 30 km to Gujranwala

↓

Remaining to Islamabad = 120 km

(Decrease is unrealistic!)

## Example




Check Consistency for edge (A, C):

$h(A) \leq c(A,C) + h(C)$

$6 \leq 2 + 4$

$6 \leq 6$ ✓ True

So, this heuristic is consistent.

## What is $c(A,C)$?

$c(A,C)$ is the **actual cost** (edge cost or step cost) of moving directly from node A to node C.




$\Rightarrow c(A,C) = 3$

It is the **real cost** of taking one step from A to C.

## Consequences of Consistency

**1) f-value never decreases along a path**

For nodes $n$ and any successor $n'$,

$f(n') = g(n') + h(n') \geq g(n) + h(n) = f(n)$

**2) A* Graph Search is optimal**

With a consistent heuristic, A* Graph Search never needs to reopen a closed node and still guarantees the **optimal** solution.

## Relationship

Consistency ⇒ Admissibility

(every consistent heuristic is admissible)

Admissibility ⇒ Consistency

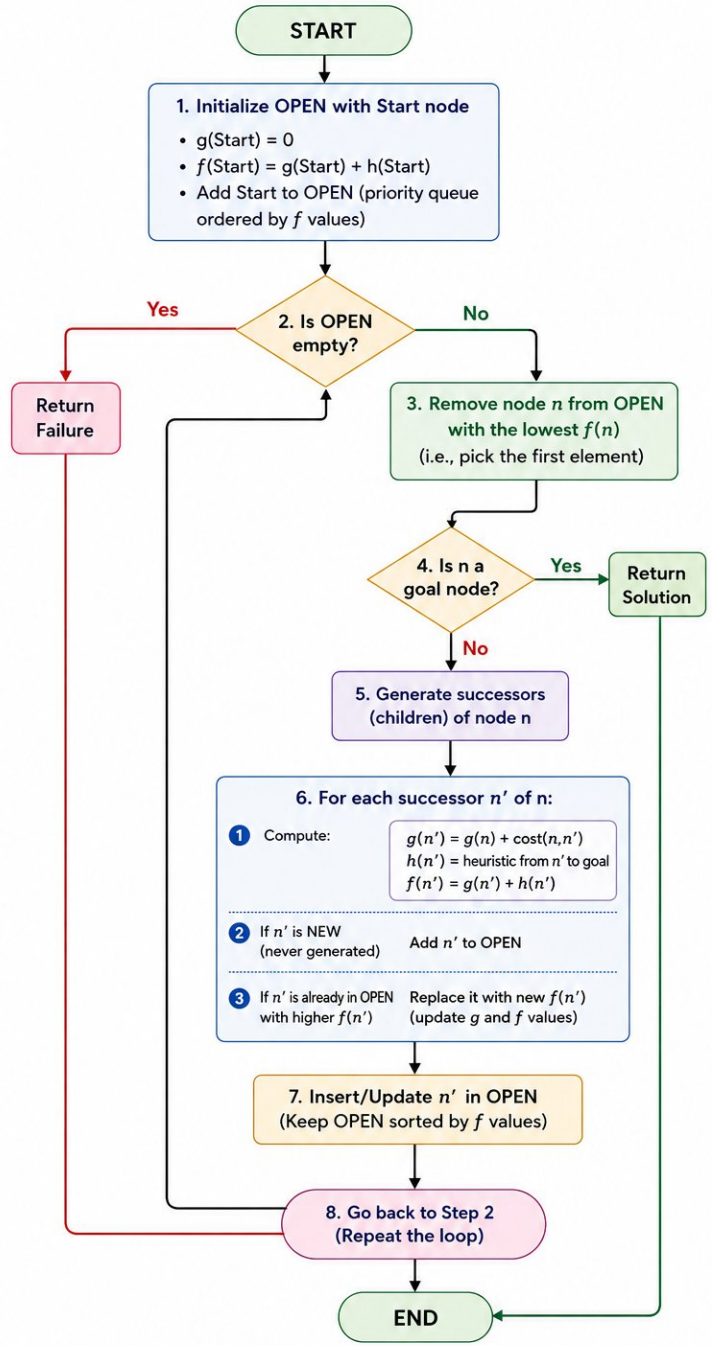
(not always true)

## Key Takeaway

Consistency ensures that heuristic estimates change smoothly from one node to the next, which makes A* both **efficient** and **optimal**.

Use consistency when you are performing **A* Graph Search**. It prevents re-expanding nodes and keeps the search **optimal**.

# A* Search Algorithm (Flowchart)




**Key**

OPEN: Frontier list (priority queue ordered by $f$ values)

$g(n)$: Cost from Start to $n$

$h(n)$: Heuristic estimate from $n$ to Goal

$f(n)$: Total estimated cost = $g(n) + h(n)$

Goal: Goal node

**What does A* do?**

Always expands the node that looks most promising (lowest $f$).

Combines actual cost so far ($g$) and estimated cost to goal ($h$).

Finds the optimal solution when the heuristic is consistent (or with Tree Search when admissible).

**Legend**

Start / End

Process

Decision

Generate / Action

Return

# A* Algorithm

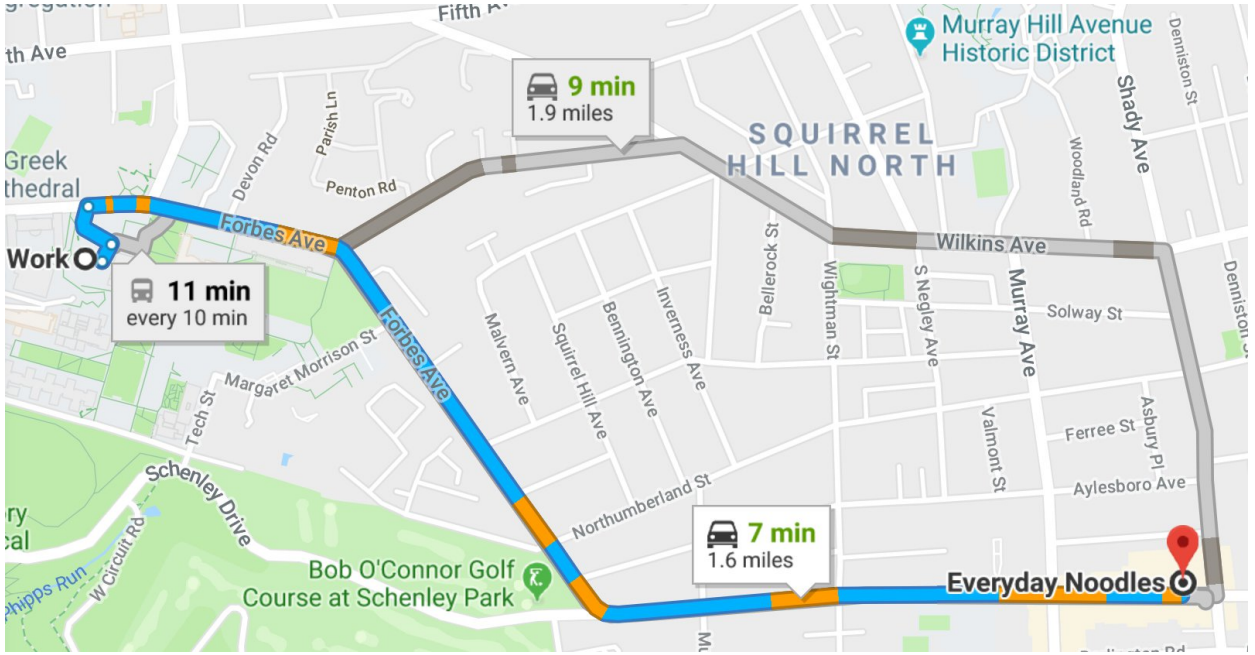
1. Search queue Q is empty.
2. Place the start state s in Q with f value h(s).
3. If Q is empty, return failure.
4. Take node n from Q with lowest f value.
   (Keep Q sorted by f values and pick the first element).
5. If n is a goal node, stop and return solution.
6. Generate successors of node n.
7. For each successor n' of n do:
   a) Compute f(n') = g(n) + h(n').
   b) If n' is new (never generated before), add n' to Q.
   c) If node n' is already in Q with a higher f value, replace it with current f(n') and place it in sorted order in Q.
   End for
8. Go back to step 3.

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
    initialize the explored set to be empty
    initialize the frontier as a priority queue using g(n) as the priority
    add initial state of problem to frontier with priority g(S) = 0
    loop do
        if the frontier is empty then
            return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then
            return the corresponding solution
        add the node state to the explored set
        for each resulting child from node
            if the child state is not already in the frontier or explored set then
                add child to the frontier
            else if the child is already in the frontier with higher g(n) then
                replace that frontier node with child
```

```
function A-STAR-SEARCH(problem) returns a solution, or failure
    initialize the explored set to be empty
    initialize the frontier as a priority queue using f(n) = g(n) + h(n) as the priority
    add initial state of problem to frontier with priority f(S) = 0 + h(S)
    loop do
        if the frontier is empty then
            return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then
            return the corresponding solution
        add the node state to the explored set
        for each resulting child from node
            if the child state is not already in the frontier or explored set then
                add child to the frontier
            else if the child is already in the frontier with higher f(n) then
                replace that frontier node with child
```

# A* Applications

- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Video games
- Machine translation
- Speech recognition
- …

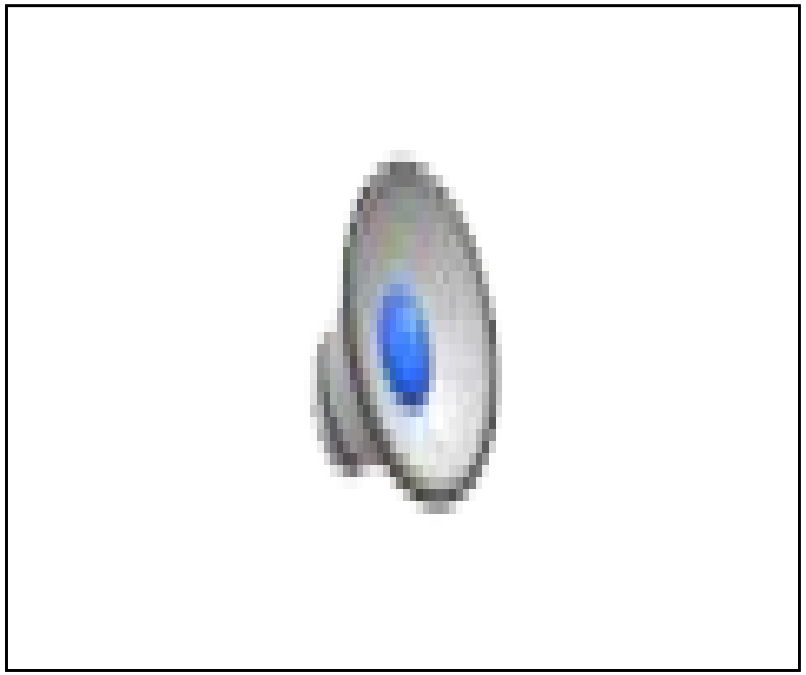


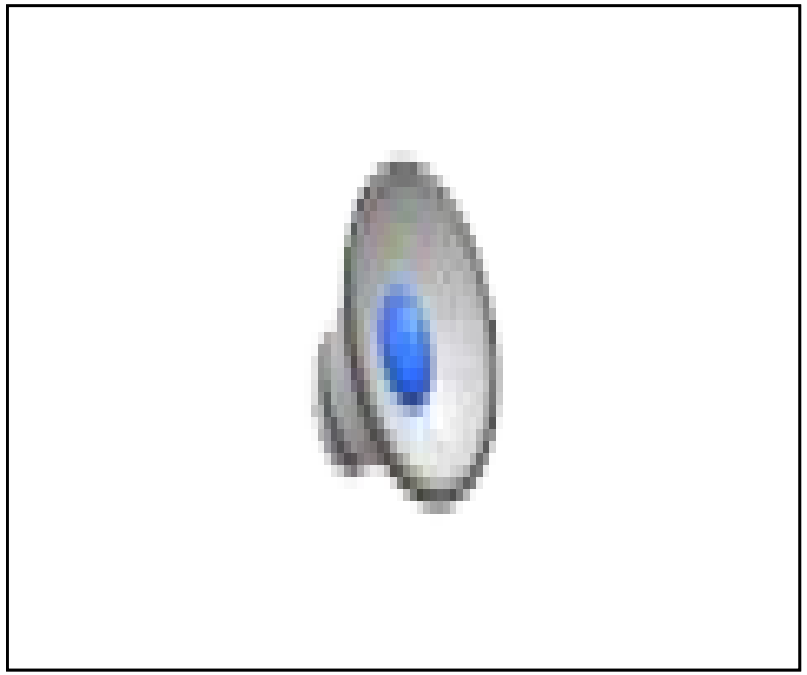
# Properties of A*

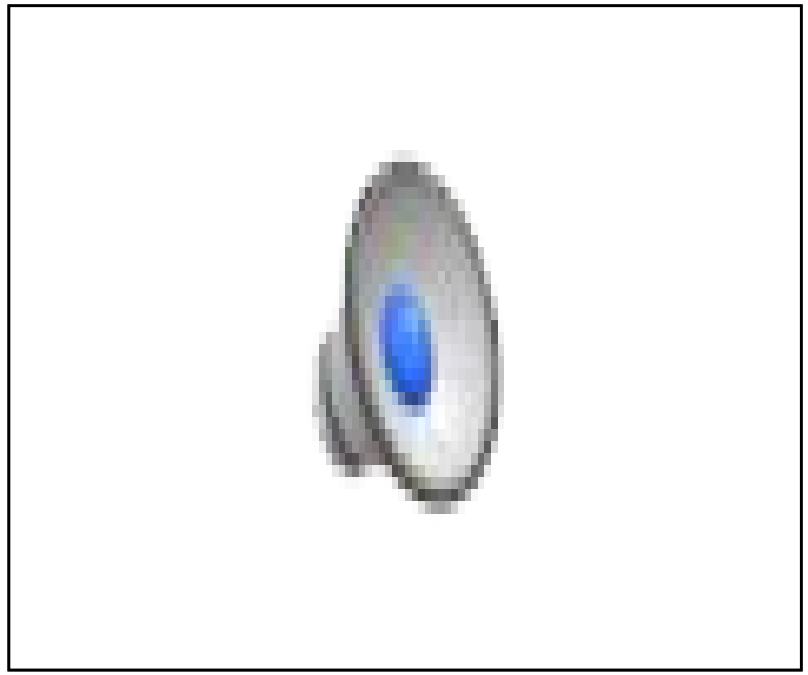
# Properties of A*

| Property | Answer | Explanation |
|---|---|---|
| **Complete?** | Yes | A* is complete if the branching factor is finite and the step costs are positive. |
| **Optimal?** | Yes | Only when the heuristic is **admissible** (Tree Search) or **consistent** (Graph Search). |
| **Time Complexity** | Exponential | Worst-case complexity is $\mathbf{O(b^d)}$. A better heuristic expands fewer nodes but the worst case remains exponential. |
| **Space Complexity** | Exponential | $\mathbf{O(b^d)}$ because A* stores all generated nodes in memory. |

# Video of Demo Contours







# Optimality

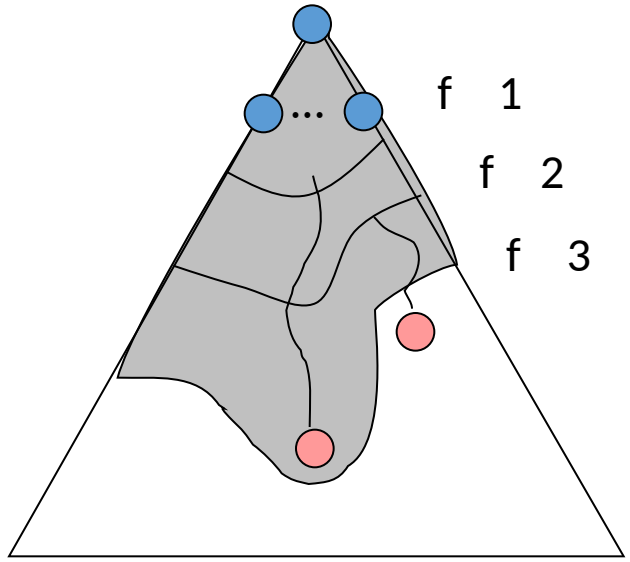
| Search Type | Condition on Heuristic | Optimal? |
|---|---|---|
| **Tree Search** | Heuristic is **Admissible** | Yes |
| **Graph Search** | Heuristic is **Consistent** | Yes |

Every consistent heuristic is admissible but not every admissible heuristic is consistent.

Consistency is stronger than admissibility

# Optimality of A* Graph Search

- Sketch: consider what A* does with a consistent heuristic:
  - Fact 1: In tree search, A* expands nodes in increasing total f value (f-contours)
  - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally



# Relationship Between Search Algorithms

*How is the next node selected?*



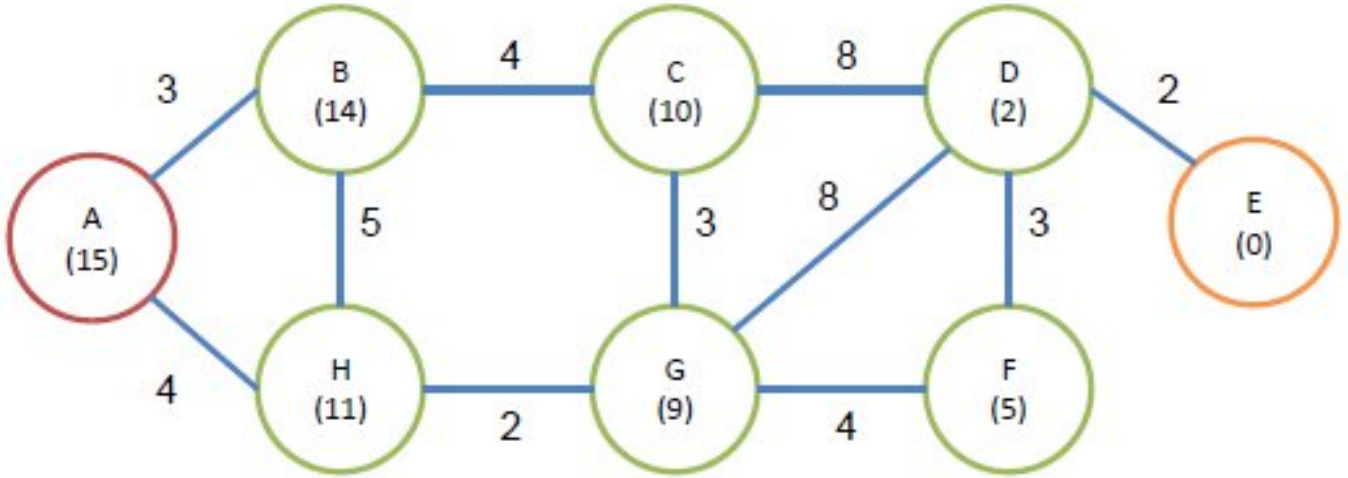

**Key Insight**

- **Uniform Cost Search:** Uses only $g(n)$ (actual cost)
- **Greedy Search:** Uses only $h(n)$ (estimated cost)
- **A* Search:** Uses $g(n) + h(n)$ to balance accuracy and guidance

* Optimal when the heuristic is **admissible** (Tree Search) or **consistent** (Graph Search).

# Question

- Perform Uniform Cost Search, Best-First Search, and the A* algorithm. Here we suppose that **A is the initial node, and E is the target**. The cost of each edge and the heuristic value of the each node are also given in the figure.



# Iterative Deepening A* Search Algorithm

# Algorithm

1. Initialization: Set the root node as the current node and find f(n) (f-score).
2. Set Threshold: Set the cost limit as a threshold (f bound) for a node i.e. the maximum f-score allowed for that node for further explorations
3. Node Expansion: Expand the current node to its children and find f-scores. Expansion only allowed if f score of node to be expanded is not greater than threshold.
4. Pruning: If for any node f-score > threshold, prune that node and store in visited node list..
5. Return Path: If Goal found, return path from start node to goal
6. Update Threshold: If goal is not found, then repeat from step 2 by changing the threshold with the minimum pruned value from visited node list and continue until goal is reached

# Iterative Deepening A* Search Algorithm

- Iterative Deepening A* (IDA*) is a graph traversal and path-finding method that can determine the shortest route in a weighted graph between a defined start node and any one of a group of goal nodes.

- A type of iterative deepening depth-first search that adopts the A* search algorithm's idea of using a heuristic function to assess the remaining cost to reach the goal.

# IDA*: Iteration 1

Current Threshold (f-bound) = **100**

**Expansion Rule:** Expand a node if $f(n) \leq 100$; Prune a node if $f(n) > 100$

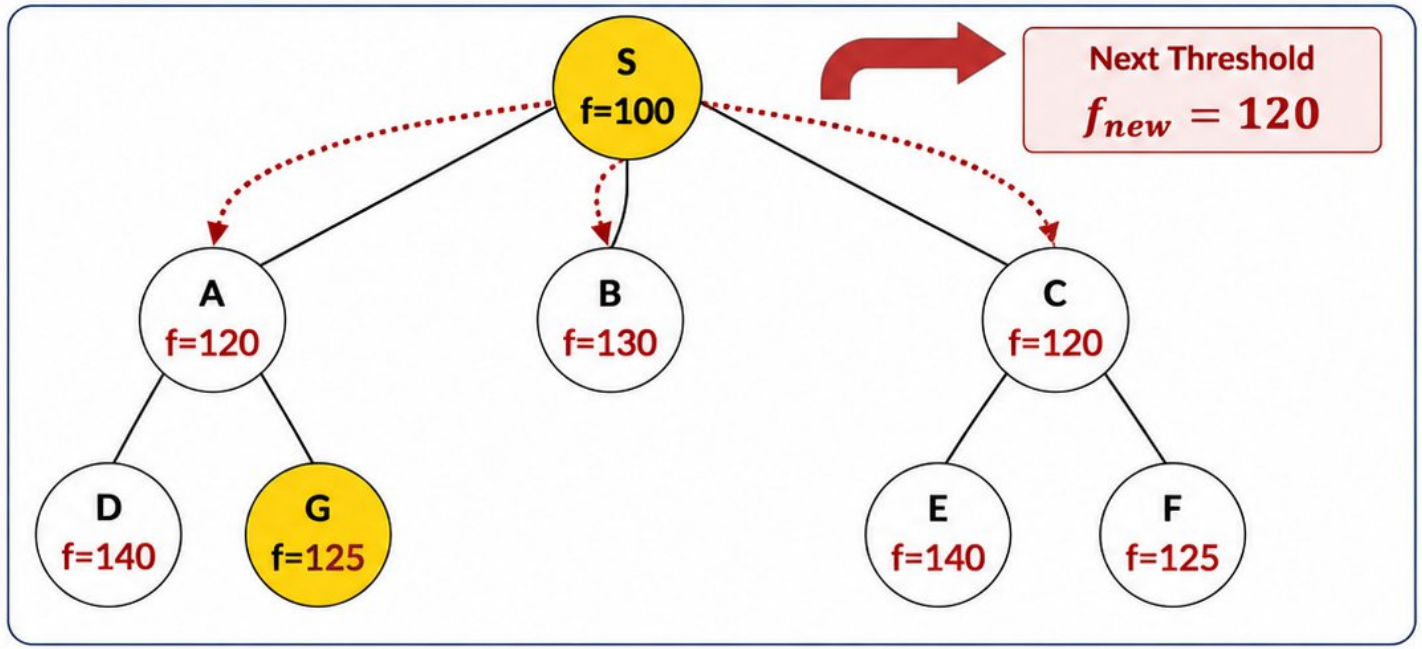
**Current Threshold**

**100**

(f-bound)

**Legend**

- Expanded ($f(n) \leq 100$)
- Pruned ($f(n) > 100$)
- Next Threshold




**What happened in Iteration 1?**

- Start node $S$ has $f(S) = 100 \leq 100 \rightarrow$ Expanded.
- All children of $S$ have $f(n) > 100 \rightarrow$ Pruned.
- No node can be expanded further.
- **We increase the threshold** to the smallest $f$-value that exceeded the current threshold.

Pruned $f$-values in this iteration

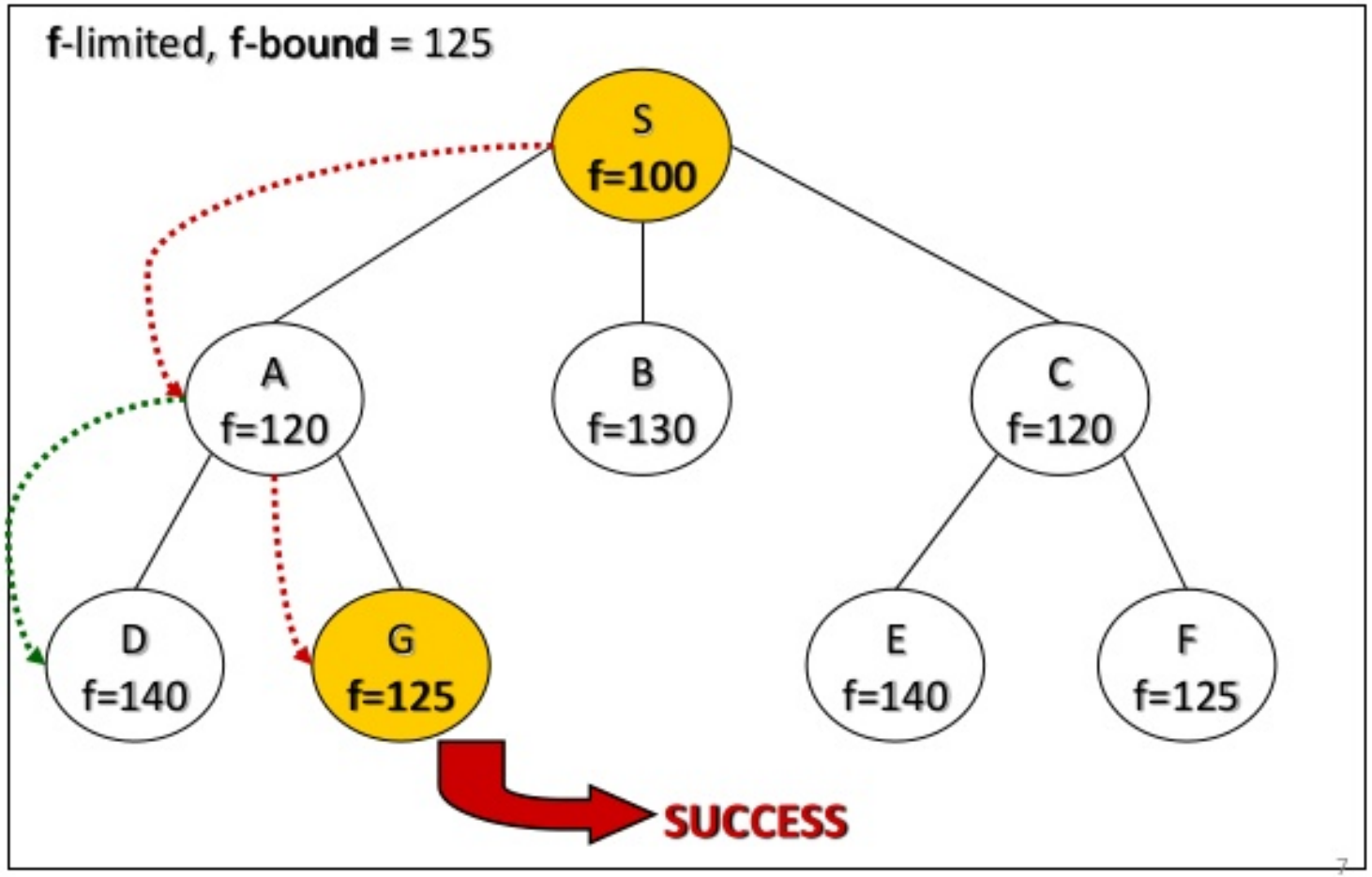
120 | 130 | 120

Smallest exceeded value = **120**

**Next Threshold ($f_{new}$) = 120**

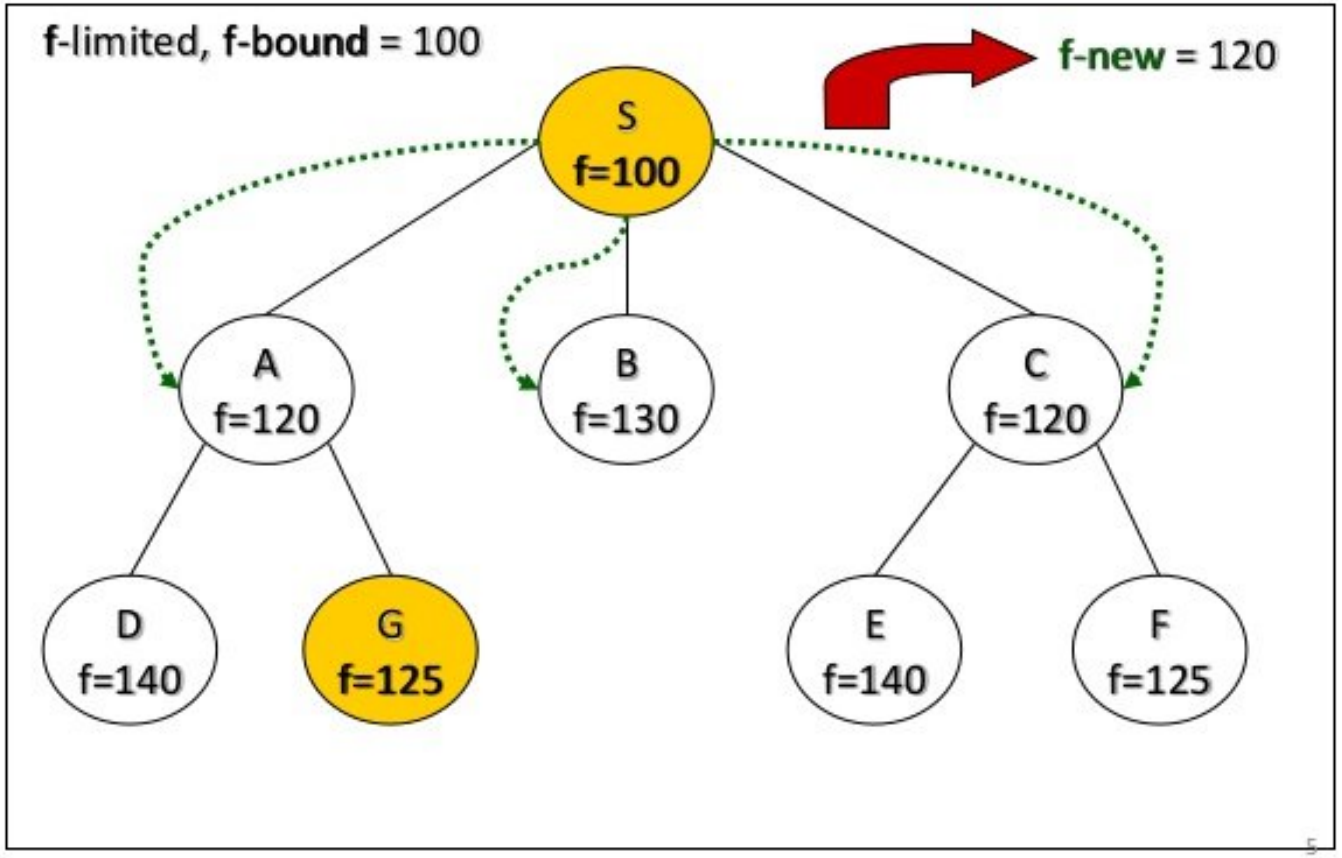
# IDA*

## Example


f-limited, f-bound = 125
S
f=100
A
f=120
B
f=130
C
f=120
D
f=140
G
f=125
E
f=140
F
f=125
SUCCESS

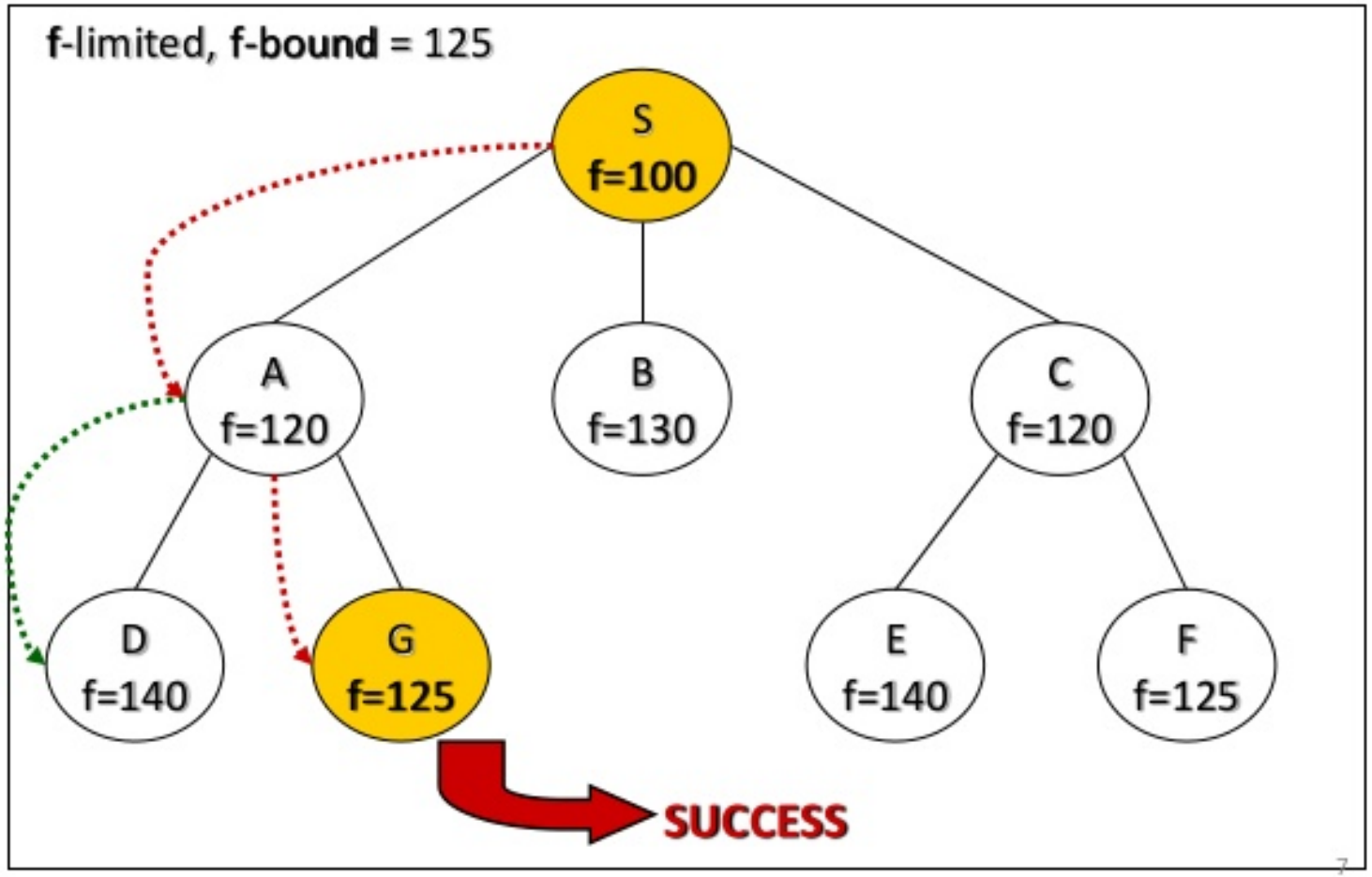
# IDA*

## Example

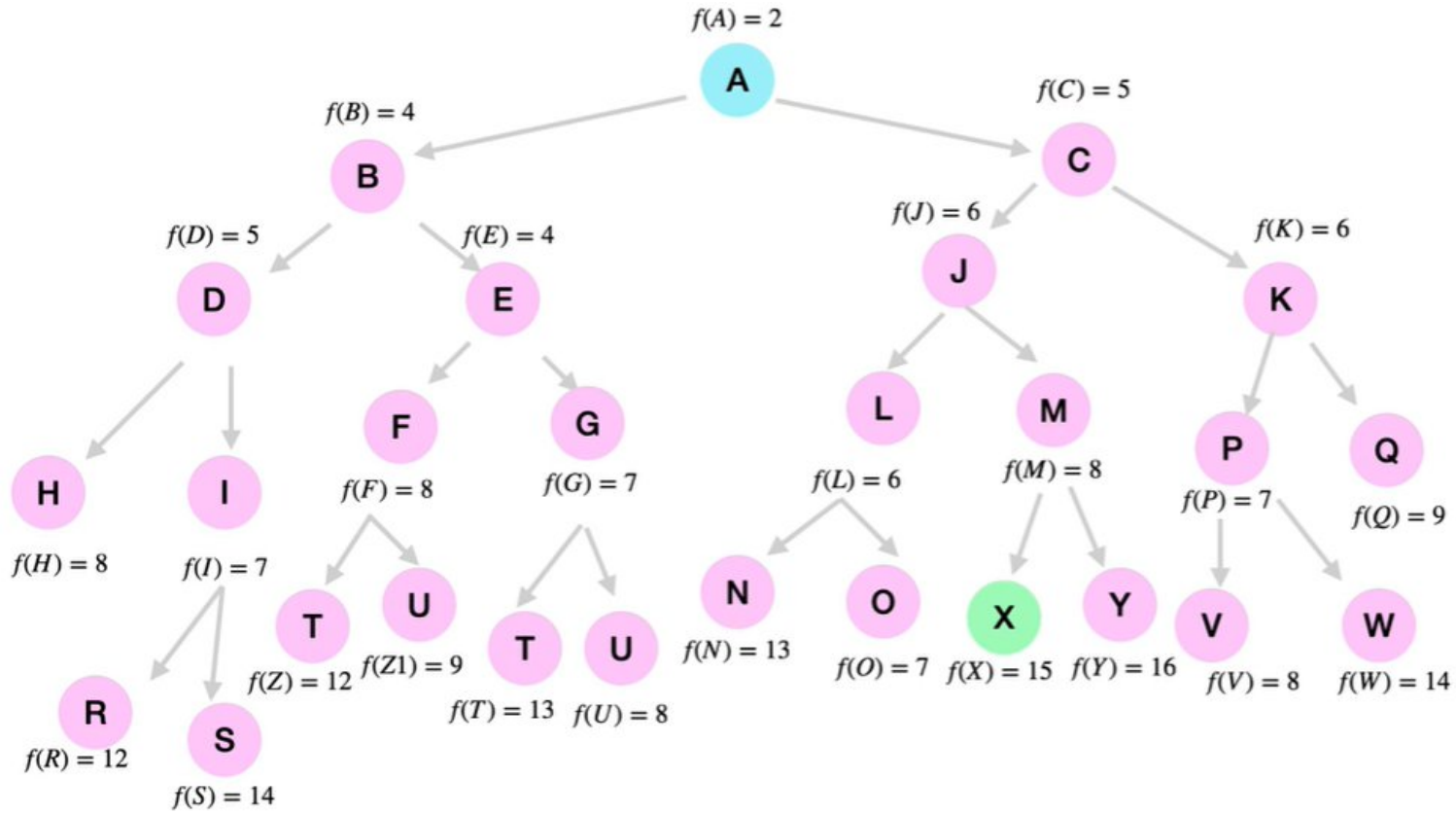

f-limited, f-bound = 100
f-new = 120
S
f=100
A
f=120
B
f=130
C
f=120
D
f=140
G
f=125
E
f=140
F
f=125

# IDA*

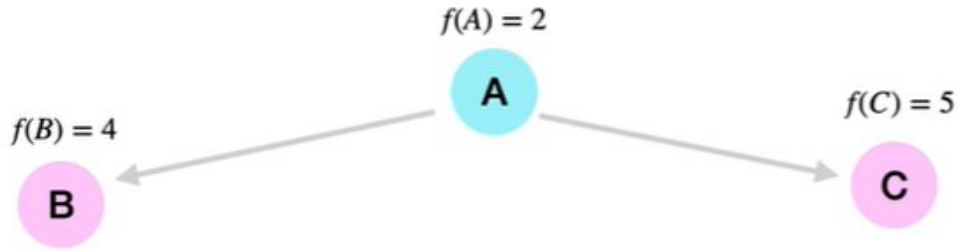
## Example


f-limited, f-bound = 125
S
f=100
A
f=120
B
f=130
C
f=120
D
f=140
G
f=125
E
f=140
F
f=125
SUCCESS

# Example:


f(A) = 2
A
f(B) = 4
B
f(C) = 5
C
f(D) = 5
D
f(E) = 4
E
f(J) = 6
J
f(K) = 6
K
H
f(H) = 8
I
f(I) = 7
F
f(F) = 8
G
f(G) = 7
L
f(L) = 6
M
f(M) = 8
P
f(P) = 7
Q
f(Q) = 9
R
f(R) = 12
S
f(S) = 14
T
f(Z) = 12
U
f(Z1) = 9
T
f(T) = 13
U
f(U) = 8
N
f(N) = 13
O
f(O) = 7
X
f(X) = 15
Y
f(Y) = 16
V
f(V) = 8
W
f(W) = 14

# Example




Iteration 1:

Current Node: A

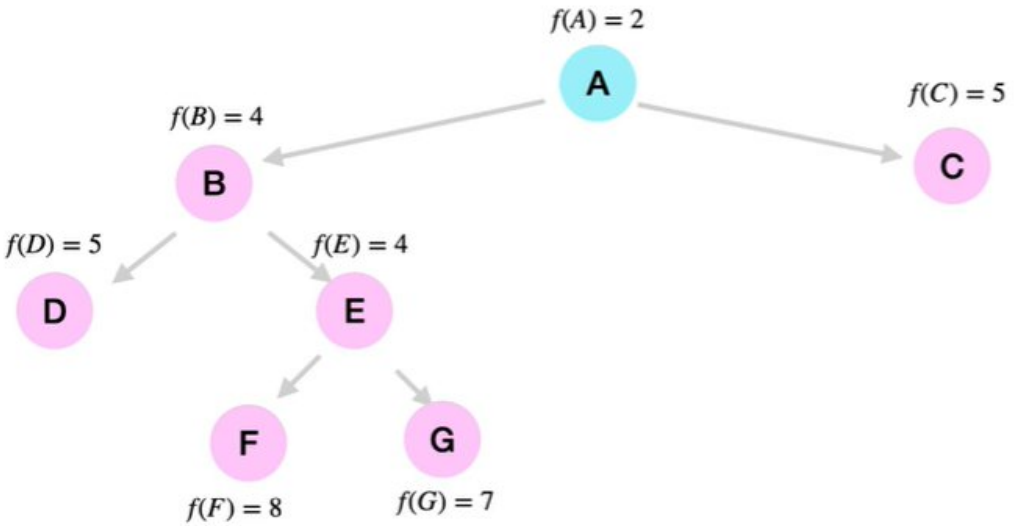
Threshold = f(A) = 2

Explore Children B, C
f(B) = 4 > Threshold 2
f(C) = 5 > Threshold 2

Both B and C pruned

Lowest pruned is B

New threshold = f(B) = 4

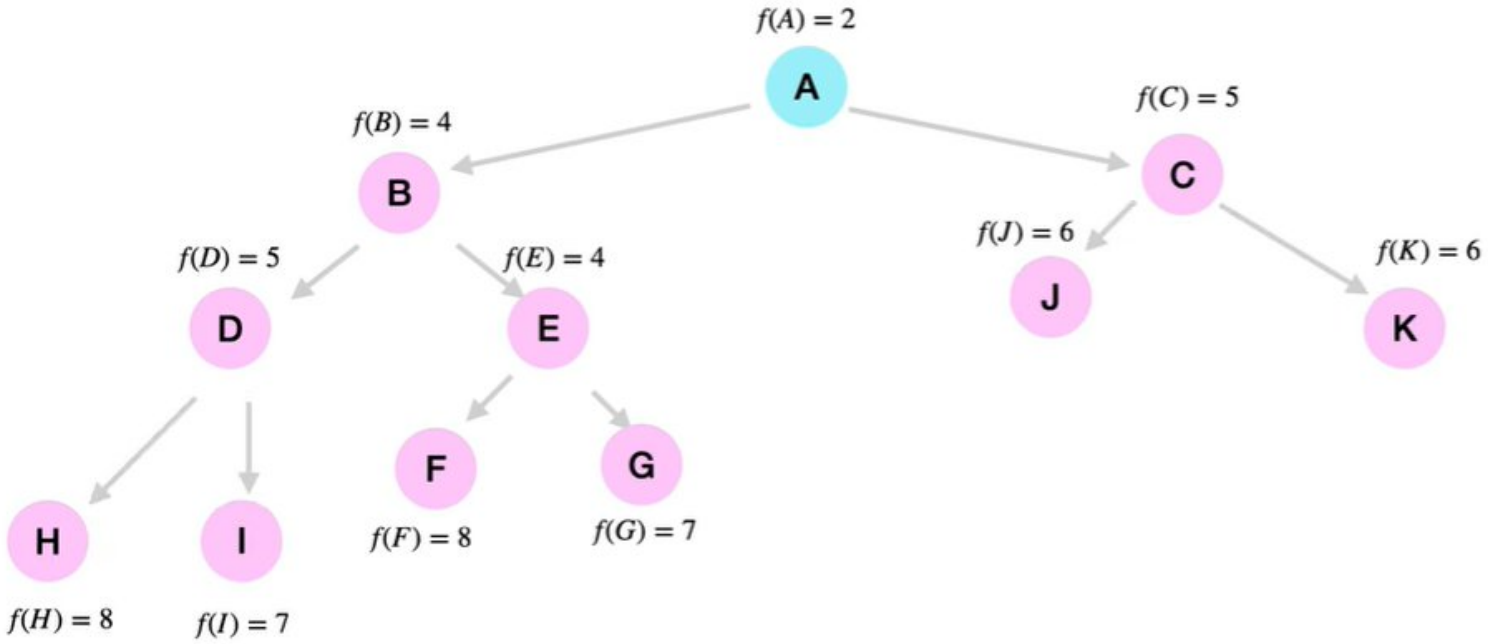



## Iteration 2:

- Current Node: A

- Threshold = 4
- f(A) = 2 < threshold

- Explore children of A: B, C
  - Current Node: B
  - f(B) = 4 = Threshold = 4
    - Explore children of B: D,E
    - f(D) = 5 > Threshold
    - Prune D
    - f(E) = 4 = Threshold
      - Explore children of E: F,G
      - f(F) & f(G) > Threshold
      - F and G are pruned
  - Current Node = C
  - f(C) > Threshold
    - Prune C
- Threshold = lowest of pruned nodes D, F, G, C= f(D) 5

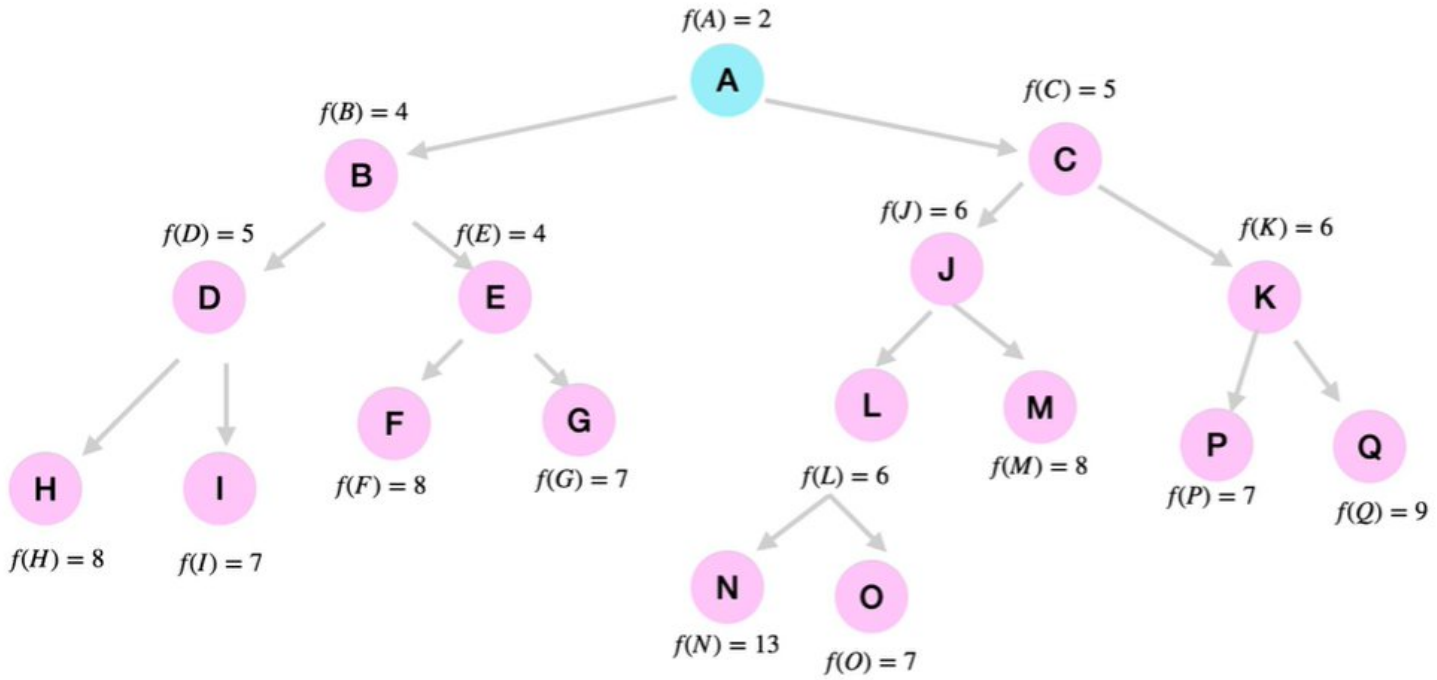



## Iteration 3:

- Current Node: A

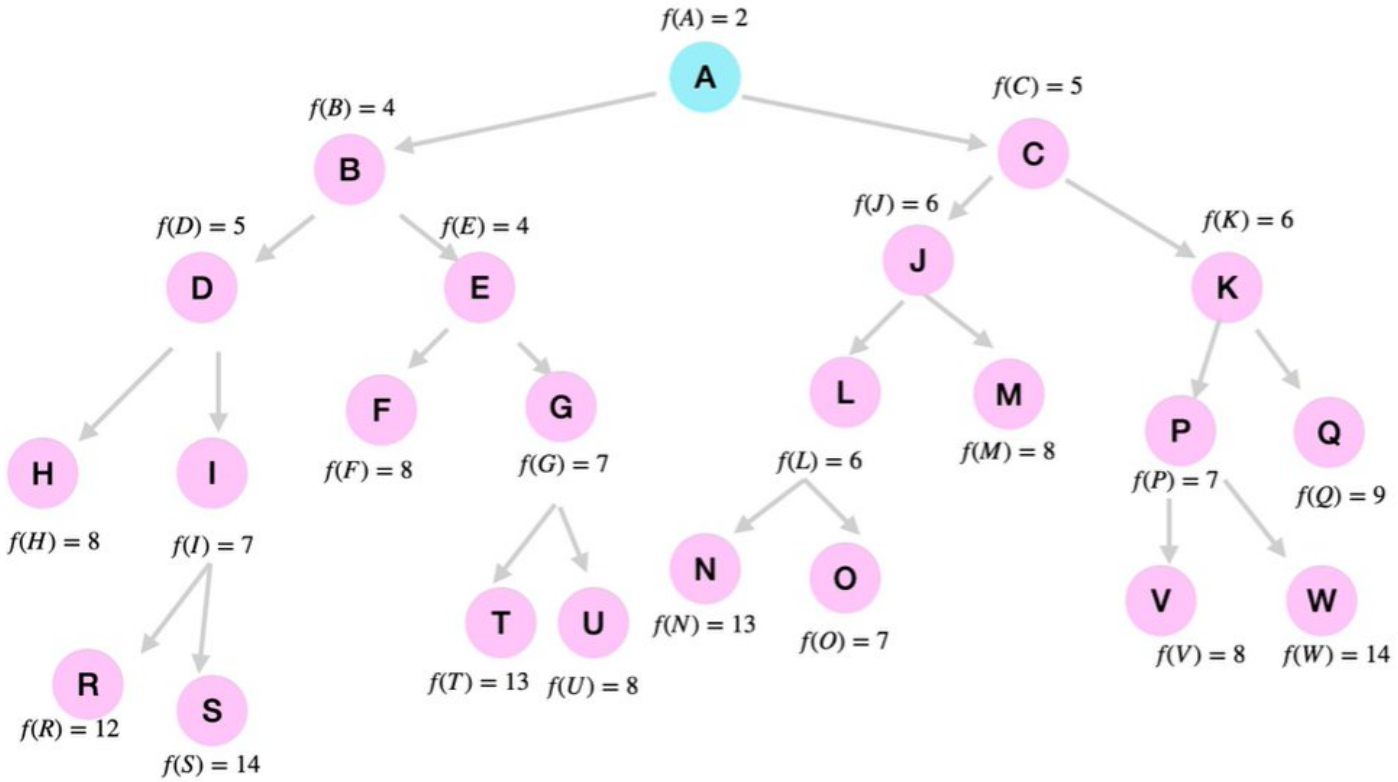
- Threshold = 5
- f(A) = 2 < threshold

- Explore children of A: B, C
  - f(B) < Threshold, f(C) = Threshold
  - Explore children of B: D,E
    - f(D) = 5 = Threshold
      - Explore children of D: H, I
        - f(H), f(I) > Threshold
        - **Prune H, I**
    - f(E) = 4 < Threshold
      - Explore children of E: F,G
        - f(F), f(G) > Threshold
        - **Prune F, G**
    - Explore children of C: J, K
      - f(J), f(K) = 6 > Threshold
      - **Prune J, K**
    - Threshold = Minimum of pruned nodes H,I,F,G,J,K = 6



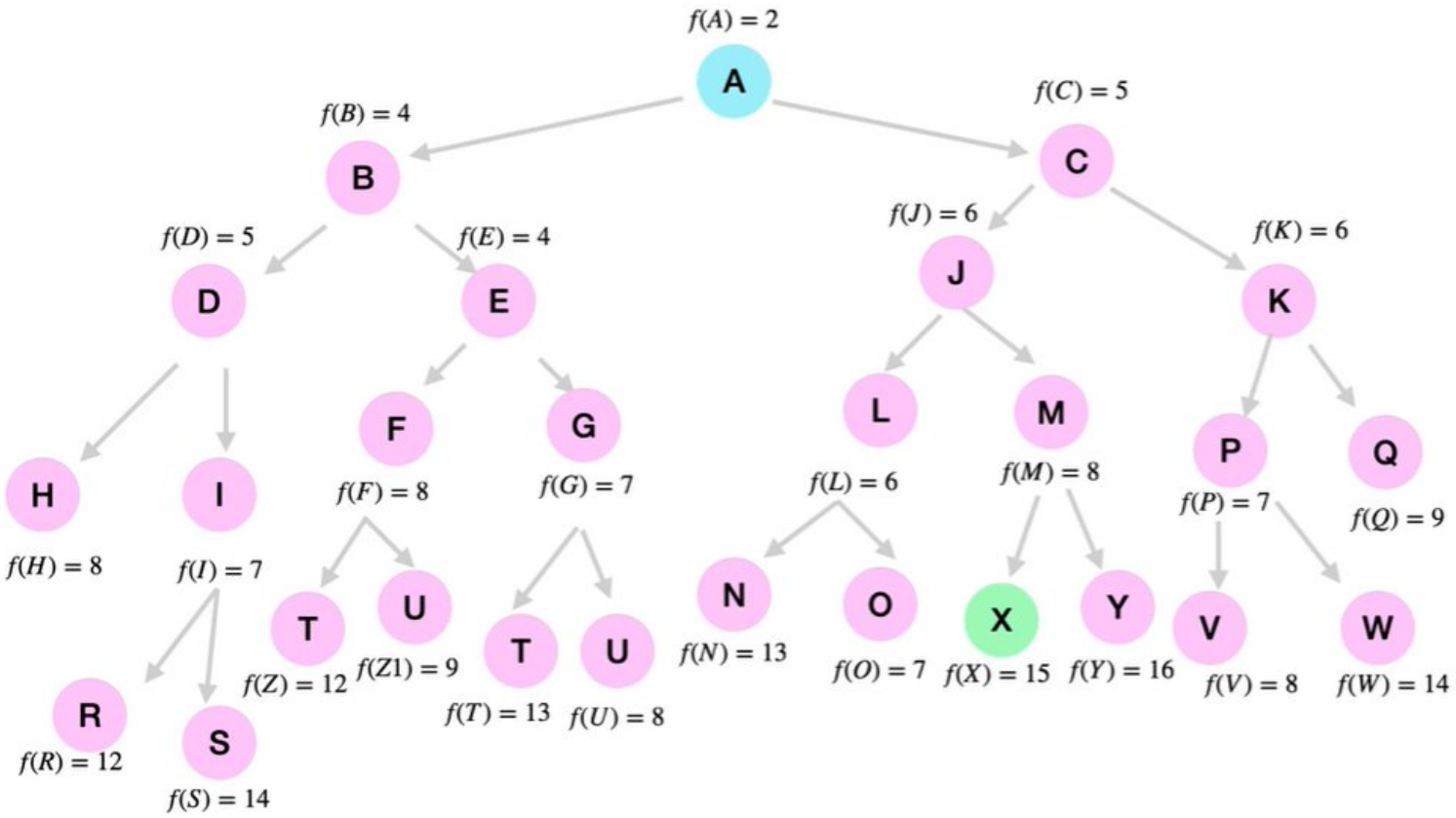

## Iteration 4:

- Current Node: A
- Threshold = 6
- f(A) = 2 < threshold
- Explore children of A: B, C
  - f(B) , f(C) < Threshold
  - Explore children of B: D,E and C: J,K
    - f(D) = 5 < Threshold
      - Explore children of D: H, I
        - f(H), f(I) > Threshold
        - **Prune H, I**
    - f(E) = 4 < Threshold
      - Explore children of E: F,G
        - f(F), f(G) > Threshold
        - **Prune F, G**
    - Explore children of C: J, K
      - f(J), f(K) = 6 = Threshold
      - Explore children of J: L,M
        - f(L) = 6 = Threshold
        - Explore children of L: N,O
          - f(N), f(O)> Threshold
          - **Prune N,O**
        - f(M) =8 > Threshold
        - **Prune M**
      - Explore children of K: P,Q
        - f(P),f(Q) > threshold
        - **Prune P,Q**
- Threshold= lowest of pruned nodes H,I,F,G,N,O,M,P,Q = 7




Iteration 5:

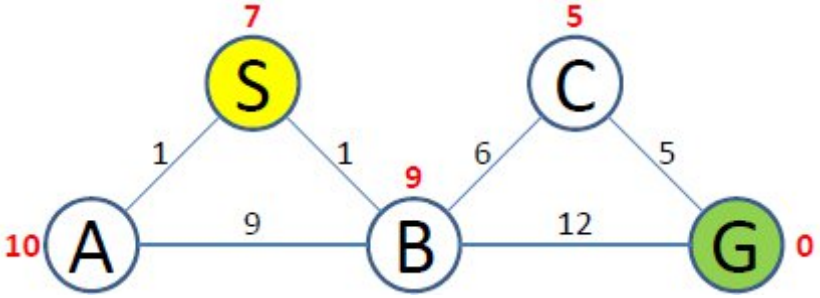
- Current Node: A
- Threshold = 7
- f(A) = 2 < threshold
- Explore children of A: B, C
  - f(B) , f(C) < Threshold
  - Explore children of B: D,E and C: J,K
    - f(D) = 5 < Threshold
      - Explore children of D: H, I
        - f(H) > Threshold
        - **Prune H**
        - f(I) = Threshold
        - Explore children of I: R,S
          - f(R) , f(S) > Threshold
          - **Prune R,S**
    - f(E) = 4 < Threshold
      - Explore children of E: F,G
        - f(F) > Threshold
        - **Prune F**
        - f(G) = Threshold
          - Explore children of G: T,U
            - f(T) , f(U) > Threshold
            - **Prune T,U**
    - Explore children of C: J, K
      - f(J), f(K) = 6 < Threshold
      - Explore children of J: L,M
        - f(L) = 6 < Threshold
        - Explore children of L: N,O
          - f(N) > Threshold
          - **Prune N**
          - f(O) = Threshold
        - f(M) =8 > Threshold
        - **Prune M**
      - Explore children of K: P,Q
        - f(P) = Threshold
        - Explore children of P : V, W
          - f(V), f(W) > Threshold
          - **Prune V,W**
        - f(Q) > threshold
        - **Prune Q**
- Threshold= lowest of pruned nodes = 8




## Iteration 6:

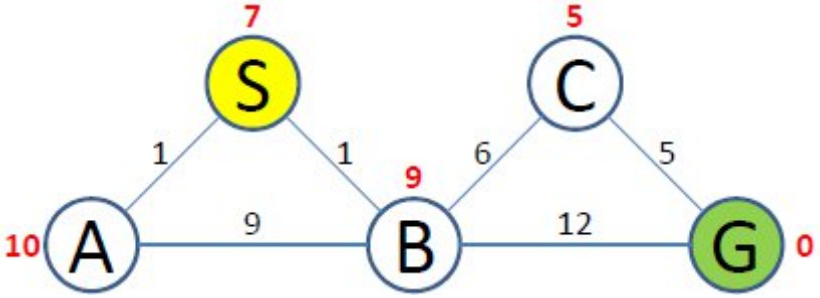
- Current Node: A
- Threshold = 8
- f(A) = 2 < threshold

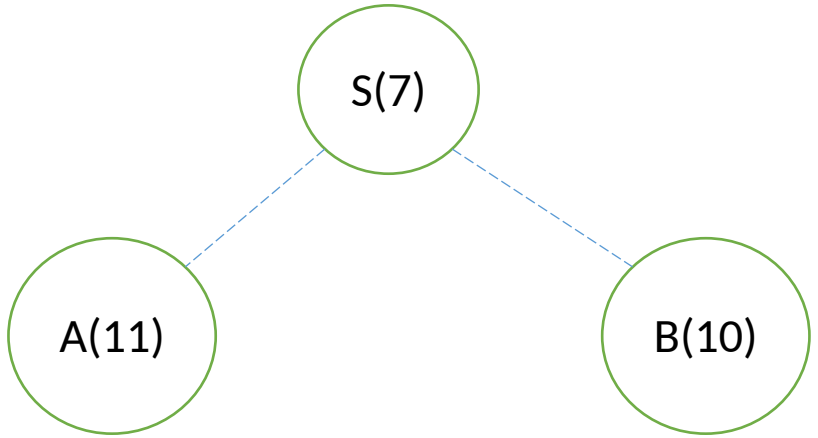
  Explore children of A: B, C
  - f(B) , f(C) < Threshold
  - Explore children of B: D,E and C: J,K
    - f(D) = 5 < Threshold
      - Explore children of D: H, I
        - f(H) = Threshold
        - f(I) = Threshold
        - Explore children of I: R,S
          - f(R) , f(S) > Threshold
          - **Prune R,S**
    - f(E) = 4 < Threshold
      - Explore children of E: F,G
        - f(F) = Threshold
        - Explore children of F: T(Z),U(Z1)
        - f(T(Z)), f(U(Z1)) > Threshold
        - **Prune T(Z), U(Z1)**
        - f(G) < Threshold
          - Explore children of G: T,U
            - f(T) >Threshold
            - **Prune T**
            - f(U) = Threshold
    - Explore children of C: J, K
      - f(J), f(K) = 6 = Threshold
      - Explore children of J: L,M
        - f(L) < Threshold
        - Explore children of L: N,O
          - f(N) > Threshold
          - **Prune N**
          - f(O) = Threshold
        - f(M) =8 = Threshold
        - Explore Children of M: X,Y
          - f(X) , f(Y) > Threshold
          - **Prune X,Y**



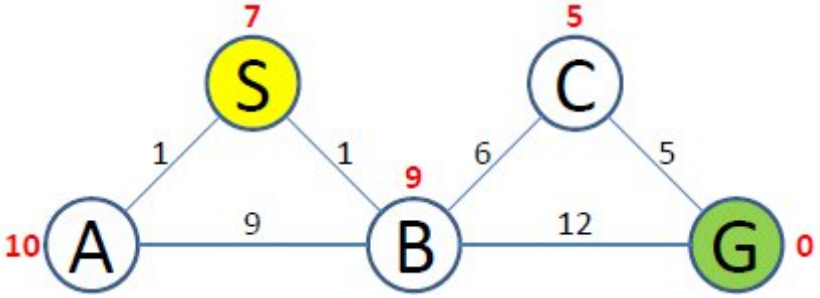

S(7)

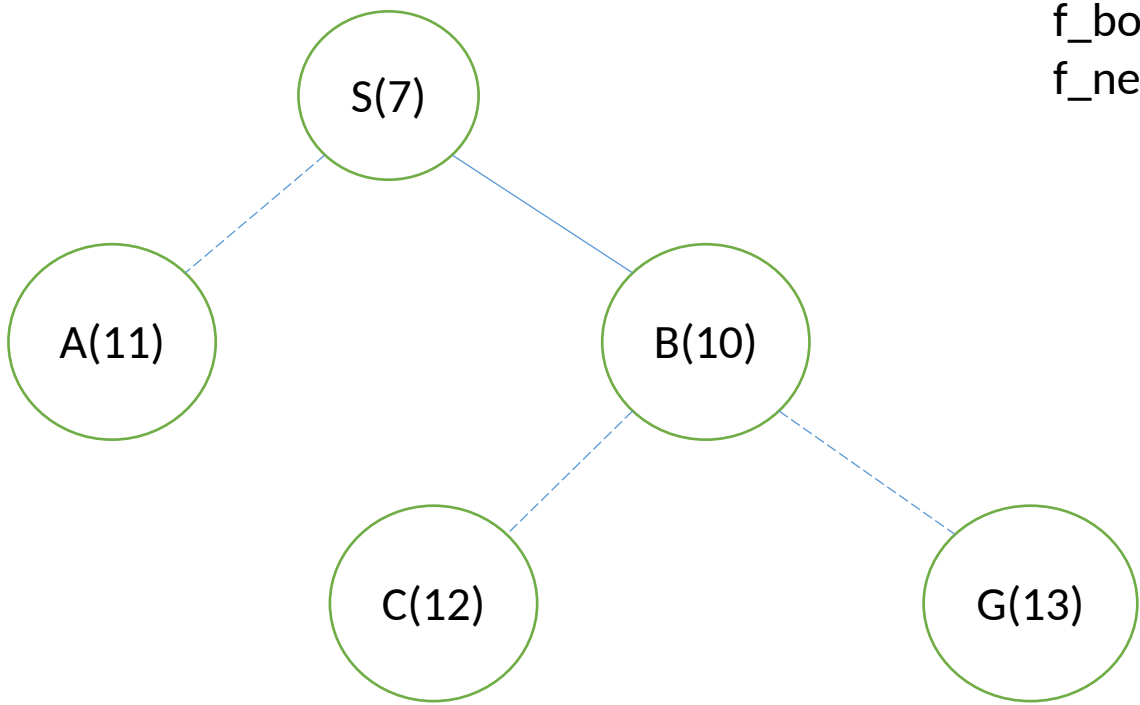
f_bound = 0
f_new = 7




f_bound = 7
f_new = 10




f_bound = 10
f_new = 11